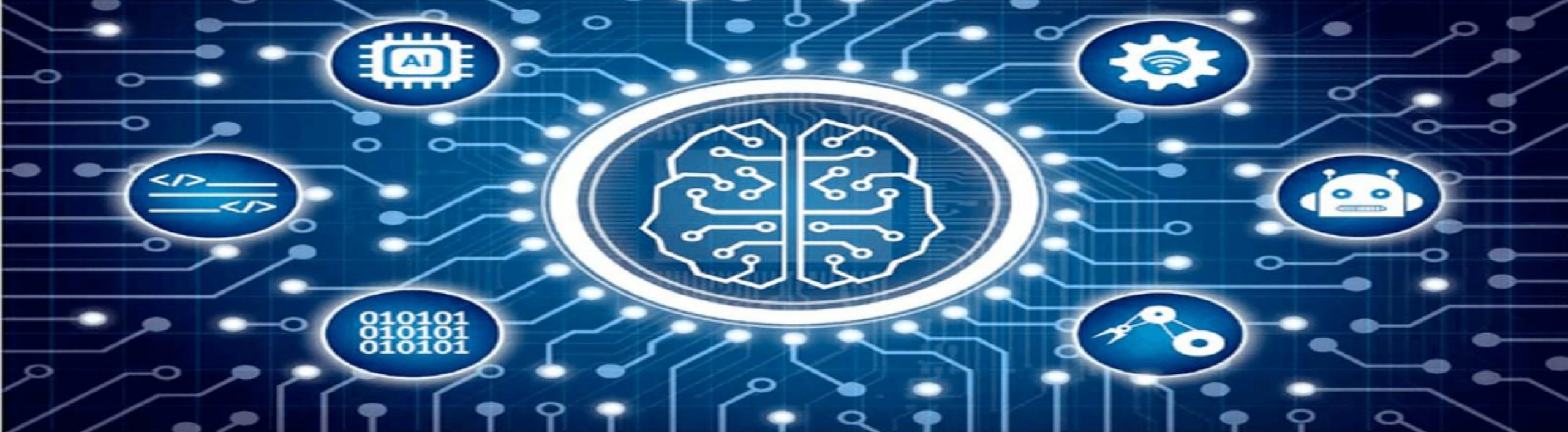




Artificial Intelligence & Machine Learning

Phase 3

With,

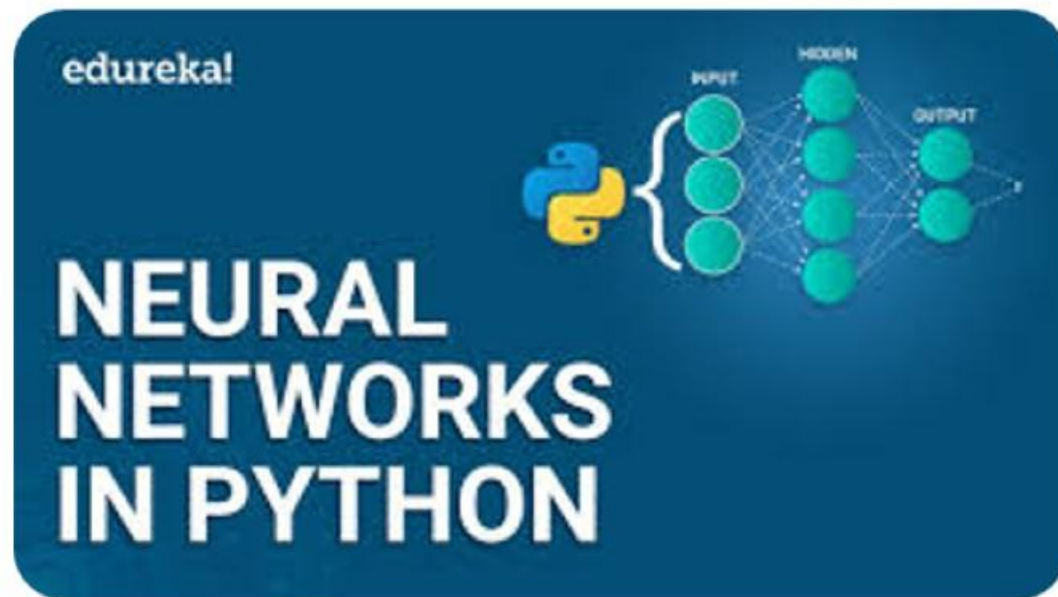
Dr. Swati Verma

L&D Consultant, Master Trainer, Capacity Enhancement Coach

Program Structure Overview

Content:

- **Weekly Themes:**
 - Week 1 → Foundations & Neural Networks
 - Week 2 → CNNs & Optimization
 - Week 3 → Transformers & NLP
 - Week 4 → LangChain, RAG, Agents
 - Week 5 → API, Deployment, Capstone
- **Daily structure:**
 - Theory (40%)
 - Hands-on Labs (60%)



IN PYTHON

Foundations & Neural Networks

Day 6

Tree Models, Ensemble Learning & Model Comparison

Week 2 - Learning Objectives



Build Advanced ML Models
Use Tree Models & Ensembles
Apply Deep Learning using Keras
Work with Text & Image AI
Understand AI Deployment & Ethics
Design Capstone AI Projects

Day 1 Learning Objectives

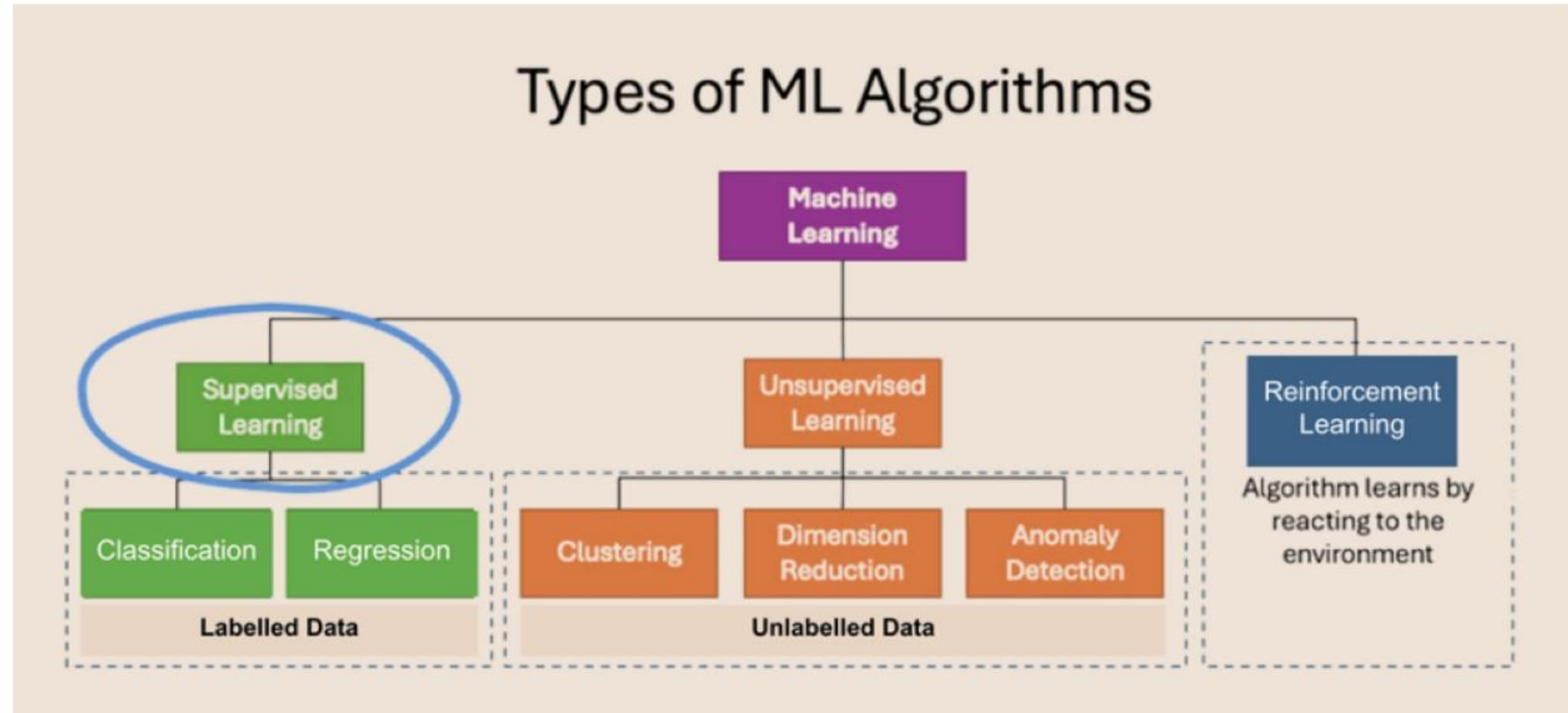


Advanced Supervised Learning	
S1 - Decision Tree	
S2 – Random Forest	
S3 – Model Comparison & Bias Forest	
Lab Objective	Defend why one model is chosen Balance accuracy vs explainability Think like ML architects, not just coders



DAY 6 — ADVANCED SUPERVISED LEARNING

ML Algorithm



SUPERVISED & UNSUPERVISED LEARNING

Two Ways to Learn from Data



Supervised Learning

Learns from data with answers.



Unsupervised Learning

Finds hidden patterns in data without answers.

SUPERVISED LEARNING

Learning with Labels



Machine learns by studying labeled fruits.



Recognizes new fruits by comparing features to what it learned.

Learning Without Labels



Machine groups unlabeled images by similar features



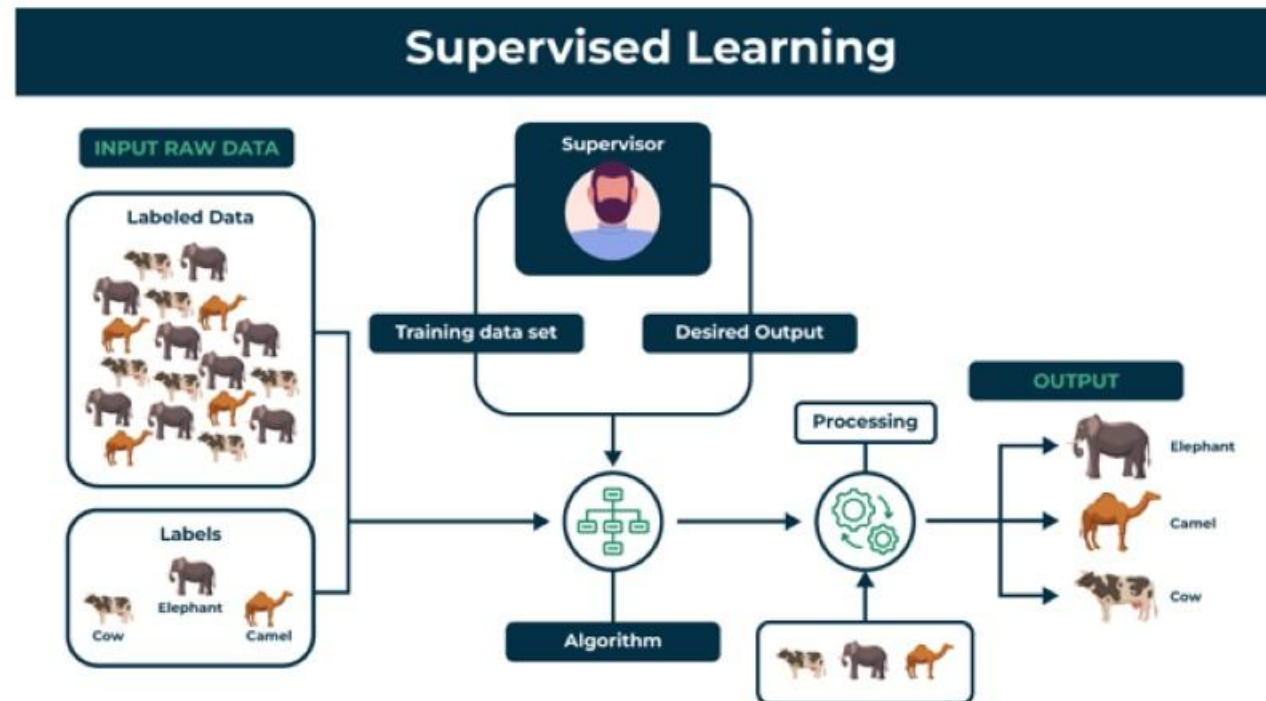
It discovers patterns without knowing the exact categories.

What is Supervised learning?

- In this approach we teach or train the machine using the labelled data(correct answers or classifications) which means each input has the correct output in the form of answer or category attached to it.
- After that machine is provided with a new set of examples (data) so that it can analyses the training data and produces a correct outcome from labeled data.

SUPERVISED LEARNING

- **For example**, a labeled dataset of images of Elephant, Camel and Cow would have each image tagged with either "**Elephant**", "**Camel**" or "**Cow**."



Types of Supervised Learning

- Supervised learning is classified into two types of algorithms:

1. Regression: A regression is used to predict **continuous** values such as house prices, stock prices or temperature. Regression algorithms learn how to connect input data to a specific number or value.

- Some common regression algorithms include:
 - Linear Regression
 - Polynomial Regression
 - Lasso Regression
 - Ridge Regression

2. Classification: A classification is used to predict **categorical** values such as whether a customer will buy or not, whether an email is spam or not or whether a medical image shows a tumor or not. Classification algorithms learn how to connect input data to the probability of belonging to different groups or categories.

- Some of the most common classification algorithms include:
 - Logistic Regression
 - Support Vector Machines
 - Decision Trees
 - Random Forests
 - Naive Baye

ADVANCED SUPERVISED LEARNING

- Advanced supervised learning" refers to sophisticated techniques and algorithms that go beyond basic regression and classification to handle complex data and improve predictive performance.
- Key aspects include advanced algorithms, ensemble methods, and specific techniques for performance enhancement.

Key Advanced Techniques and Algorithms

- **Ensemble Methods:** These techniques combine multiple individual models (often called "weak learners") to create a more powerful "strong learner".
 - **Random Forests:** Multiple decision trees are trained on different subsets of data and features, and their results are aggregated (e.g., via majority voting for classification) to make a final, more robust prediction.
 - **Boosting (Gradient Boosting, AdaBoost, XGBoost):** This iterative approach builds models sequentially, with each new model correcting the errors made by the previous ones.

XGBoost is a popular and efficient variant known for its accuracy and speed.

- **Support Vector Machines (SVMs):** Highly effective for complex, high-dimensional datasets, SVMs work by finding the optimal "hyperplane" that maximizes the margin between different data classes. The use of kernel functions (e.g., RBF) allows them to handle non-linear relationships.

Key Advanced Techniques and Algorithms

- **Neural Networks (Deep Learning):**

- Multi-layer perceptrons (MLPs) and deeper architectures like Convolutional Neural Networks (CNNs) for image tasks and Transformer models for natural language processing (NLP) are used for highly complex, unstructured data.

- **Semi-supervised and Self-supervised Learning:**

- These bridge the gap between supervised and unsupervised learning by using a small amount of labeled data along with a large amount of unlabeled data to automate the labeling process and scale effectively, particularly useful when manual labeling is expensive.

Performance Enhancement Techniques

- **Regularization:**

- Techniques like L1 (Lasso) and L2 (Ridge) regularization add penalty terms to the loss function to prevent **overfitting**, ensuring the model generalizes well to new, unseen data.

- **Cross-Validation:**

- Methods like k-fold cross-validation are used for robust model evaluation and hyperparameter tuning. This ensures the model's performance is accurately estimated across different data partitions.

- **Feature Engineering and Selection:**

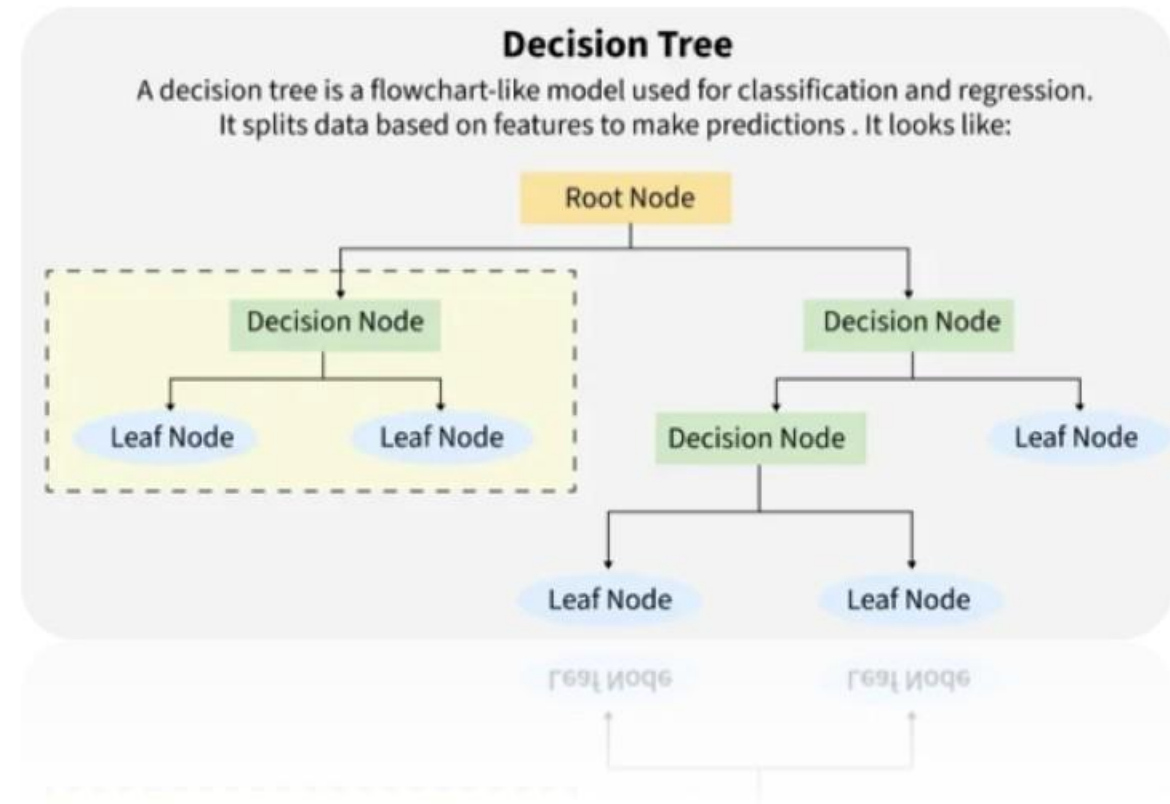
- This involves extracting the most relevant features from the raw data or creating new ones to improve model performance and reduce complexity.

- **Optimization Algorithms:**

- Advanced optimizers like Adam (Adaptive Moment Estimation) are used during training to efficiently update model parameters and minimize the loss function.

SESSION 1 — DECISION TREE

- A **Decision Tree** is a supervised ML algorithm that:
 - Makes decisions using a **tree-like structure**
 - Splits data based on **Yes/No rules**
 - Ends with a **final decision (prediction)**
- It works exactly like:
 - “If this happens → do this, otherwise → do that”



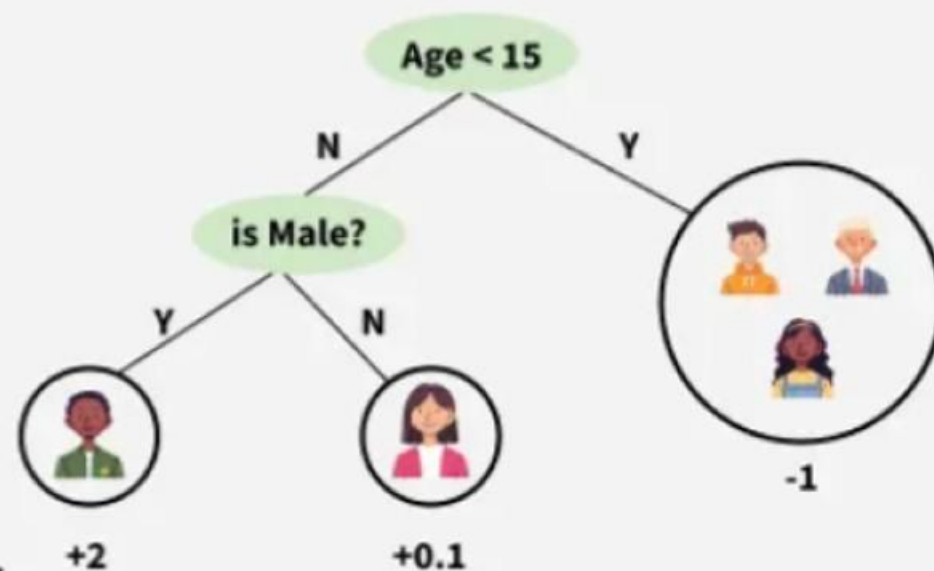
SESSION 1 — DECISION TREE

Working of Decision Tree

The model checks conditions like age and gender to split users into groups. Each group (leaf node) gets a prediction score based on user preferences for computer games.

Input: Age, Gender, Occupation,...

Does the person likes computer games



Prediction score in each leaf

Where is Decision Tree Used?

Industry	Use Case
Banking	Loan approval
Insurance	Claim approval
HR	Employee promotion
Healthcare	Disease diagnosis
Retail	Customer segmentation

How Decision Tree Makes Decisions

- It works using:
 - Data Splitting
 - Rules at each node
 - Final prediction at leaf node
- Example:
 - IF Income > 50,000 →
 - IF CreditScore > 700 → Loan Approved
 - Else → Reject
 - Else → Reject

Splitting Criteria

- Decision Trees select the **best question** using:

Metric	Purpose
Gini Index	Measure impurity
Entropy	Measure randomness
Information Gain	Best feature selector

Overfitting vs Underfitting (Important Interview Topic)

Problem	Meaning
Overfitting	Model memorizes training data
Underfitting	Model too simple

- Solution:
 - Control **tree depth**
 - Use **min_samples_split**

LAB 1A — DECISION TREE (LOAN APPROVAL SYSTEM)

-  **Objective**
 - Build a **Loan Approval Prediction System** using Decision Tree.
-  **Parameters Used:**

Feature	Meaning
Income	Monthly Income (₹)
CreditScore	CIBIL Score
Experience	Years of work
LoanApproved	Target (1 = Yes, 0 = No)

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import
train_test_split
from sklearn.metrics import
classification_report, confusion_matrix
```

LAB 1A — DECISION TREE (LOAN APPROVAL SYSTEM)

-  **Objective**
 - Build a **Loan Approval Prediction System** using Decision Tree.
-  **Parameters Used:**

Feature	Meaning
Income	Monthly Income (₹)
CreditScore	CIBIL Score
Experience	Years of work
LoanApproved	Target (1 = Yes, 0 = No)

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import
train_test_split
from sklearn.metrics import
classification_report, confusion_matrix
```


LAB 1A — DECISION TREE (LOAN APPROVAL SYSTEM)

- **IMPORTING REQUIRED LIBRARIES**

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
```

Library

pandas

DecisionTreeClassifier

train_test_split

confusion_matrix

classification_report

Why We Use It

To create and manage tabular data (like Excel)

To build our ML decision-making model

To divide data into training & testing

To check how many predictions are right/wrong

To calculate Precision, Recall, F1-Score



LAB 1A — DECISION TREE (LOAN APPROVAL SYSTEM)

- **1 IMPORTING REQUIRED LIBRARIES**

- **2 CREATING THE LOAN DATASET**

- **3 SEPARATING INPUT & OUTPUT**

```
X = df[["Income", "CreditScore", "Experience"]]
```

```
y = df["LoanApproved"]
```

- **4 TRAIN-TEST SPLIT (MOST IMPORTANT STEP)**

- **`X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)`**

Part

`X_train, y_train`

`X_test, y_test`

`test_size=0.3`

Purpose

Used to **teach the model**

Used to **test real-world accuracy**

70% Training, 30% Testing



LAB 1A — DECISION TREE (LOAN APPROVAL SYSTEM)

- **5** BUILDING THE DECISION TREE MODEL
 - The model learns rules like:
 - If **CreditScore > 720 AND Income > 60k** → Approve
 - If **Low income + low experience** → Reject
 - `model = DecisionTreeClassifier()`
 - `model.fit(X_train, y_train)`
- **6** MODEL MAKES PREDICTIONS

`y_pred = model.predict(X_test)`

 - The model now predicts loan approval for **test customers it has never seen before.**



LAB 1A — DECISION TREE (LOAN APPROVAL SYSTEM)

- **7** CONFUSION MATRIX (CORE EVALUATION)
 - **`print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))`**

Term	Meaning
True Positive (TP)	Correctly approved
True Negative (TN)	Correctly rejected
False Positive (FP)	Wrongly approved
False Negative (FN)	Wrongly rejected

- This tells the bank:
 - How many risky customers were wrongly approved
 - How many safe customers were wrongly rejected



LAB 1A — DECISION TREE (LOAN APPROVAL SYSTEM)

8 CLASSIFICATION REPORT (ACCURACY METRICS)

```
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Metric

Business Meaning

Precision

How trustworthy approvals are

Recall

How many good customers we did not miss

F1-Score

Overall balance

Accuracy

Overall correctness



LAB 1B — Employee Promotion Eligibility (HR Analytics)

- **Objective:**

Predict whether an employee should be promoted based on:

- Experience
- Performance Rating
- Certifications
- Attendance

- **Business Outcome:**

Data-driven promotion decision system.



LAB 1B (ENHANCED) — EMPLOYEE PROMOTION ELIGIBILITY SYSTEM (HR ANALYTICS)

-  **BUSINESS OBJECTIVE (FOR SLIDE)**
 - Build an AI-based HR promotion eligibility system that evaluates employees using performance, experience, certifications, attendance, and leadership potential — and justifies promotion decisions.

- **Output**

Experience	Performance	Certifications	Attendance	LeadershipScore	Promoted	Model_Prediction	Promotion_Status	HR_Criteria_Status
2	3	0	70	2	0	0	Not Promoted	Criteria Not Fulfilled
5	4	2	85	3	1	1	Promoted	Criteria Fulfilled
8	5	4	92	4	1	1	Promoted	Criteria Fulfilled
3	3	1	76	2	0	0	Not Promoted	Criteria Not Fulfilled
10	5	5	95	5	1	1	Promoted	Criteria Fulfilled



LAB 1B (ENHANCED) — EMPLOYEE PROMOTION ELIGIBILITY SYSTEM (HR ANALYTICS)

- User Input (HR Manager enters employee details)
- AI Promotion Prediction
- HR Policy Criteria Check
- Decision Tree Visualization (Nodes + Rules)
- Full Tabular Output

LAB 1C — Insurance Policy Acceptance (Insurance Sector)

- **Objective:**

Decide whether an insurance customer is **High Risk / Low Risk** using:

- Age
- Health Score
- Smoking Status
- Accident History

- **Business Outcome:**

Risk-based premium & policy issuance.



RANDOM FOREST

SESSION 2

What is Random Forest?

- Random Forest is an **ensemble ML model** that:
 - Builds **multiple decision trees**
 - Combines their outputs
 - Produces a **stronger, stable prediction**

● Why Random Forest is Powerful

- Reduces overfitting
- Higher accuracy
- Handles missing data
- Works well for real business datasets

Bagging Concept

- **1. Bagging = Bootstrap + Aggregation**
- **Meaning:**
 - Train trees on random samples
 - Combine all outputs
- **2. Random Feature Selection**
 - Each tree sees **random columns**
 - Reduces bias
 - Increases generalization
- **3. Voting System**
 - If 70 out of 100 trees say “Fraud” → Final output = Fraud

Random Forest Vs Decision Tree

Feature	Decision Tree	Random Forest
No. of Trees	1	Many
Stability	Low	High
Overfitting	High risk	Very low
Speed	Fast	Slower
Accuracy	Medium	High
Business Use	Rule engines	Mission-critical systems

WHY BIG COMPANIES USE RANDOM FOREST

Industry	Application
Banking	Fraud detection
Telecom	Churn prediction
Insurance	Claim risk
HR	Attrition
Healthcare	Disease detection
E-Commerce	Customer behavior

REAL BUSINESS ANALOGY

- Decision Tree = One Doctor Diagnosis
Random Forest = Medical Board Discussion
- Decision Tree = One Loan Officer
Random Forest = Credit Risk Committee

WHEN TO USE AND NOT USE RANDOM FOREST

- **WHEN SHOULD YOU USE RANDOM FOREST?**

- When accuracy is critical
- When risk is high
- When data is noisy
- When features are many
- When wrong decisions cost money

- **WHEN SHOULD YOU NOT USE RANDOM FOREST?**

-  When:
 - Data is very small
 - Speed is more important than accuracy
 - Explainability is legally required

OUTPUT TYPES OF RANDOM FOREST

Task Type	Output
Classification	Class (Fraud / Genuine)
Regression	Numeric value
Probability	Risk score



LAB 2A — FRAUD DETECTION USING RANDOM FOREST (BANKING)

-  **Objective**
- Detect **fraudulent transactions** using Random Forest.
 - Amount
 - LocationMismatch
 - DeviceMismatch
 - PastFraud
 - Fraud

```
from sklearn.ensemble import RandomForestClassifier
```

FRAUD CRITERIA (WHAT BANKS ACTUALLY USE)

Feature	Meaning	Fraud Logic
Amount	Transaction Value	Very high amount → Risk
LocationMismatch	IP vs Home Location	1 = Different country
Hour	Transaction Time	Late night → Suspicious
DeviceTrusted	Known Device	0 = Unknown device
FailedAttempts	OTP Failures	More attempts → Fraud



DAY 7 — ADVANCED SUPERVISED LEARNING

RANDOM FOREST Cont....

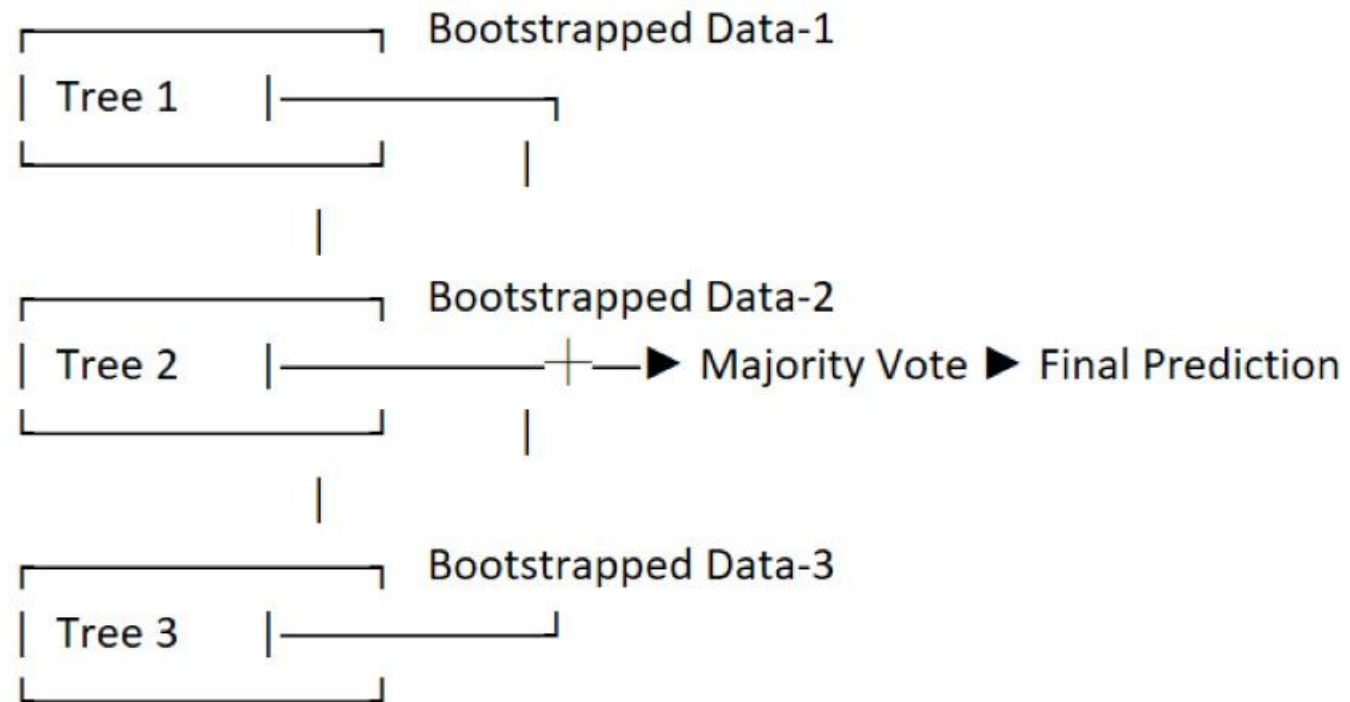
Day 1 Learning Objectives



Advanced Supervised Learning - Done	
S1 - Decision Tree - Done	
S2 – Random Forest CONTINUE..	
S3 – Model Comparison & Bias Forest	
Lab Objective	Defend why one model is chosen Balance accuracy vs explainability Think like ML architects, not just coders

Random Tree

Random Forest Concept



CODING CHECKLIST

- **✓ Step 1 — Prepare dataset**

- Identify **input features (X)**
- Identify **target label (y)**

- **✓ Step 2 — Train-test split**

- Always split data so the evaluation is fair

- **✓ Step 3 — Build Random Forest**

```
from sklearn.ensemble import  
RandomForestClassifier
```

```
model = RandomForestClassifier(  
    n_estimators=100,    # number of  
    trees                # depth of trees  
    max_depth=None,     # reproducible  
    random_state=42      results  
)
```

CODING CHECKLIST

- **✓ Step 4 — Fit the model**

```
model.fit(X_train, y_train)
```

- **✓ Step 5 — Predict**

```
y_pred = model.predict(X_test)
```

- **✓ Step 6 — Evaluate**

```
print(classification_report(y_test,  
y_pred))  
print(confusion_matrix(y_test,  
y_pred))
```

- **✓ Step 7 — User Input (Business Case)**

- Always allow participants to input new data.

- **✓ Step 8 — OPTIONAL**

- View **feature importance** to see what factors matter.

```
print(model.feature_importances_)
```

Mini Example Code

```
from sklearn.ensemble import  
RandomForestClassifier  
import pandas as pd
```

```
data = {  
    "Age": [25, 45, 35, 60, 40, 50],  
    "BP": [120, 150, 130, 160, 125, 140],  
    "Cholesterol": [180, 220, 200, 260, 190, 230],  
    "Disease": [0, 1, 0, 1, 0, 1]  
}
```

```
df = pd.DataFrame(data)
```

```
X = df.drop("Disease", axis=1)  
y = df["Disease"]
```

```
model =  
RandomForestClassifier(n_estimators=5)  
model.fit(X, y)  
  
print(model.predict([[50, 150, 220]]))
```


Random Forest – Tree Concept

- A tree is **NOT** “Age tree” or “BP tree.”
- Instead:
 - ✓ Each tree uses **ALL** columns (Age, BP, Cholesterol)
- BUT...
- **! Each tree sees:**
 - Different random rows (bootstrapping)
 - Different random subsets of columns (feature bagging)
- So Tree 1 may use different data than Tree 2.

Random Forest – Tree Concept

Age	BP	Cholesterol	Disease
25	120	180	0
45	150	220	1
35	130	200	0
60	160	260	1
40	125	190	0
50	140	230	1



Summary

- **No, each tree is not a tree of Age or a tree of BP.**
 - Each tree uses **all features but with randomness**:
 - Random rows
 - Random subset of columns
 - Random splits
 - Random thresholds
- This is why Random Forest is powerful and stable.

Random Forest – Tree Concept

-  **Random Forest = Many Trees but Each Tree Is Like a Different Person Learning Differently**
- **Tree 1 gets (random rows):**
 - Rows: 1, 3, 5, 2
 - Columns randomly allowed: Age + BP
- **Tree 2 gets (random rows):**
 - Rows: 2, 6, 6, 3
 - Columns randomly allowed: BP + Cholesterol
- **Tree 3 gets (random rows):**
 - Rows: 4, 1, 2, 5
 - Columns randomly allowed: Age + Cholesterol
- **✓ So Tree 1, Tree 2, Tree 3 all learn different patterns**
 - That's why they don't become same or overfit.
-  **Final Prediction = Majority Vote**
- **Example for new patient:**
 - Tree 1 → High Risk
 - Tree 2 → Low Risk
 - Tree 3 → High Risk
- **Final Output = High Risk**

LAB 2B — Medical Disease Risk Prediction (Healthcare)

- **Objective:**

Predict **High Disease Risk** using:

- Age
- BP
- Sugar Level
- BMI

- **The system must work like a hospital self-assessment module:**

- Patients can enter values
- Model outputs probability
- System gives medical recommendations

- **Business Outcome:**

Early warning system for hospitals.

LAB 2C — Customer Default Prediction (NBFC / Lending)

- **Objective:**

Predict **Loan Default Probability** using:

- Salary
- EMI Burden
- Credit History
- Spending Score

- **Business Outcome:**

Risk control engine for lending firms.

- The model should be: Customer Self-

Check tool

- ✓ Predict whether the applicant may default
- ✓ Give a probability score
- ✓ Interpret the score (Low / Medium / High risk)
- ✓ Provide NBFC business recommendations
- ✓ Accept new customer input

Feature	Meaning
Income	Monthly income in INR
CreditScore	CIBIL-like score
LoanAmount	Active loan amount
EMI_to_Income	EMI / Income ratio
Prev_Defaults	No. of previous defaults
Default (Target)	1 = Will default, 0 = Safe



MODEL COMPARISON & BIAS—VARIANCE

SESSION 3

Why Compare Models?

- Different models behave differently on:
 - Small data
 - Noisy data
 - Complex patterns

Bias vs Variance

Type	Meaning
High Bias	Too simple → Underfitting
High Variance	Too complex → Overfitting

Model Comparison Summary

Model	Strength
Logistic Regression	Simple & explainable
Decision Tree	Rule based
Random Forest	High accuracy

LAB 3A - MODEL COMPARISON (LOGISTIC VS TREE VS FOREST)

- **OBJECTIVE**

- To compare the performance of multiple Machine Learning models on the same business dataset and recommend the most suitable model for real-world deployment based on accuracy, stability, and business risk.

- **Tasks**

- Train **3 different ML models** on the same dataset
- Measure and compare their **prediction accuracy**
- Understand how **complex models outperform simple models**
- Learn **how businesses choose production models**
- Develop **model selection decision-making skills**

LAB 3A - MODEL COMPARISON (LOGISTIC VS TREE VS FOREST)

- Choose any Lab data to compare

Problem	Dataset to use	Target
Loan Approval	Lab 1A	LoanApproved
HR Promotion	Lab 1B	Promoted
Health Risk	Lab 2B	HighRisk
Customer Default	Lab 2C	Default
Fraud Detection	Lab 2A	FraudFlag

★ Which Score Should We Rely On To Decide the Best Model?

- For **binary classification** like Health Risk, Default, Fraud, Churn, Promotion, etc. — we should compare the following:
-  **1. Accuracy**
 - “How many predictions were correct overall?”
 - ✓ Good when classes are balanced (e.g., 5 low risk, 5 high risk)
 - ✗ Not reliable if dataset is imbalanced
-  **2. Precision**
 - “When the model says HIGH RISK, how often is it correct?”
 - **Business use:**
 - High Precision = fewer false alarms
 - Hospitals prefer high precision for high-risk alerts
-  **3. Recall (Sensitivity / TPR)**
 - “When the patient is actually HIGH RISK, how often did the model catch them?”
 - **Business use:**
 - High Recall = fewer missed risky patients
 - Critical in healthcare and fraud

★ Which Score Should We Rely On To Decide the Best Model?

- 🟪 4. F1 Score (Balanced Precision + Recall)
- A single score combining precision & recall.
 - **Business use:**
 - MOST IMPORTANT when both classes matter
 - BEST SCORE to compare models

- ★ **FINAL RULE**
- **“Use F1-Score as the final**

decision metric for comparing Logistic Regression, Decision Tree, and Random Forest.”

- **Because:**
 - Accuracy can lie
 - Recall may be high but precision low
 - Precision may be high but recall low
 - F1 balances both



LAB 3B — Churn Prediction Model Selection (Retail / Telecom)

- **Objective:**

Compare:

- Logistic Regression
- Decision Tree
- Random Forest

on customer churn dataset.

- **Business Outcome:**

Retention strategy engine.

- A Telecom/Retail company wants to identify customers who might **churn** (leave the service or stop buying).

- You have customer attributes such as:

- Tenure
- Monthly Spend
- Complaints Count
- Payment Delay
- Usage Drop
- Churn Flag (0 = No, 1 = Yes)

- Your job:

Train multiple models → Compare them → Choose the best one using F1 Score.



LAB 3B — Churn Prediction Model Selection (Retail / Telecom)

- **✓ Step 1 — Load realistic dataset**

- This reflects real churn drivers:
 - High complaints → churn
 - High usage drop → churn
 - High monthly spend + short tenure → high churn risk
 - Payment delay → churn

-

- **✓ Step 2 — Train 3 Models**

- Logistic Regression checks for linear patterns
- Decision Tree splits customers into segments
- Random Forest creates many trees to stabilize prediction

- **✓ Step 3 — Evaluate**

- Rely on:
 - ✓ Accuracy
 - ✓ Precision
 - ✓ Recall
- **F1 Score (MOST IMPORTANT)**

- **✓ Step 4 — Choose the best model**

- Your code prints:
 - **BEST MODEL IS: Random Forest (most cases)**

LAB 3C-1 — Credit Risk Model Selection (Banking)

- **Objective:**

Find the best-performing model for:

- Credit approval
- Default prediction
- EMI failure risk

- **Business Outcome:**

Core credit bureau modeling system.

- **Business Use Case**

- A bank wants to build a **Credit Risk Model** to classify borrowers into:

- **0 = Low Risk**

- **1 = High Risk (Possible Default)**

- Features include:

Feature	Meaning
Income	Monthly income (INR)
CreditScore	CIBIL-like score
LoanAmount	Total loan exposure
EMI_to_Income	EMI-to-Income ratio
PreviousDefaults	Count of past missed payments

★ What is SHAP in Machine Learning?

- *(SHAP = SHapley Additive exPlanations)*
- SHAP is a **model explainability technique** used to understand **why** an ML model made a particular prediction.
- **SHAP** tells you how much each feature (Income, CreditScore, BP, Tenure, etc.) contributed to the final prediction — positively or negatively.
- It is widely used in:
 - **Banking & Credit Risk** (RBI/Regulatory requirement)
 - **Healthcare AI**
 - **Insurance Underwriting**
 - **Fraud Detection**
 - **HR Analytics (Promotion, Attrition)**



What is SHAP in Machine Learning?

- Example:
 - A model predicts “**High Risk = 0.82 probability**”
 - SHAP will show:
- Meaning:
 - EMI ratio increased risk a lot
 - Credit score also increased risk
 - Higher income decreased risk
 - This is a **complete explanation**, not a black box.

Feature	SHAP Impact
EMI Ratio	+0.35
Low Credit Score	+0.30
Income	-0.10
Past Defaults	+0.22
Age	+0.05

★ Why SHAP is Important

- ✓ Makes ML models transparent
- ✓ Helps business users trust ML
- ✓ Required in BFSI (as per model audit guidelines)
- ✓ Helps interpret feature importance for every prediction
- ✓ Used in interviews for Data Science roles
- ✓ Helps debug the model (bias, feature errors, etc.)

When Should You Use SHAP?

- ✓ You need to explain **why a customer is high risk**
- ✓ Bank asks for **model audit**
- ✓ You want to show **feature impact** instead of just prediction
- ✓ Business needs **interpretability**



SHAP explains the contribution of each input feature towards a prediction, making machine learning models interpretable and trustworthy.

What SHAP Actually Tells Us

- **SHAP does NOT tell you which model is best.**
SHAP tells you:
“Why did the model make this prediction?”
- Think of SHAP as:
 - **X-Ray of a Machine Learning Model**
 - Accuracy = “How good is the doctor?”
 - SHAP = “What symptoms made the doctor give this diagnosis?”
-  **Model selection = Accuracy-based**
-  **Interpretation/explainability = SHAP-based**

Why SHAP Is Needed in BFSI, Healthcare, HR

- Regulators ask:
 - *Why was this loan rejected?*
 - *Why is this customer high-risk?*
 - *Why did the model classify this patient as severe?*
- Models cannot say “Because accuracy = 92%”.
They must show:
 - → **What features contributed**
 - → **How much they contributed**
 - → **Whether the decision was fair**

SHAP in Health Risk Example

- Let's say the model predicts:
- **Patient: 82% High Risk (diabetes/heart issue)**
- SHAP gives:

Feature	Effect (SHAP Value)	Meaning
Sugar 220	+0.45	Major reason for high risk
BMI 33	+0.30	Moderate contributor
Age 45	+0.05	Small contributor
BP 130	-0.10	Reduces risk

Model Comparison - Labs Assignment

- ✓ LAB 3C-2 — Feature Importance (Which features drive credit risk?)
- ✓ LAB 3C-3 — SHAP Explainability (Used in real BFSI)
- ✓ LAB 3C-4 — Probability Threshold Tuning (Bank policy tuning)
- ✓ LAB 3C-5 — Customer input + Risk recommendation engine

LAB 3C-3 — Credit Risk Explainability with SHAP (BFSI Standard)

- **Objective**

- Build a credit risk model and **explain every prediction using SHAP**, as required in banking regulatory guidelines & model audit.

- **Use Case**

- Bank wants to understand:
 - “**Why** is a customer predicted as **High Default Risk**?”
 - Which features contributed towards it?
 - Is the model explainable for internal

audit?

- BFSI interpretation rules
 - **+ SHAP value = Increases risk**
 - **– SHAP value = Reduces risk**
 - **Values close to 0 = Neutral**
 - **Sum of SHAP values = final model prediction log-odds**

Learning Summary

✓ Logistic Regression

- Best for **simple linear relationships**
- Works well when features are numeric and linearly separable
- NOT good when the data has complex patterns

✓ Decision Tree

- Easy interpretability
- Works with non-linear data
- Can overfit (memorizes training data)

✓ Random Forest

- Best accuracy
- Handles non-linearity
- Very stable
- Used in real-world fintech + healthcare + HR analytics



DAY 7 — UNSUPERVISED LEARNING (ADVANCED)

Learning without labels | Hidden patterns | Segmentation | Anomaly Detection

WHAT IS UNSUPERVISED LEARNING?

- **Unsupervised Learning is a machine learning approach where the algorithm learns patterns from data without any labeled output.**
- **Simple Meaning**
- No “Correct Answer” column
- Model finds:
 - Groups
 - Structures
 - Similarities
 - Outliers

Unsupervised Learning – Discovering Hidden Intelligence in Data

- Unsupervised Learning is a powerful AI method where algorithms find hidden patterns, structures, and insights in unlabeled data without human guidance, exploration and feature extraction, transforming raw data into actionable knowledge.
 - Acting like a data detective to group similar items (clustering), find rules (association), or spot anomalies,
 - Crucial for tasks like customer segmentation, fraud detection, and understanding complex datasets.
 - It excels at discovering unknown relationships, making it ideal for
- **Core Concept**
 - **No Labels:** Unlike supervised learning (which uses tagged data), unsupervised models learn from data that has no predefined answers or categories.
 - **Self-Discovery:** The algorithm infers its own rules and structures, identifying similarities, differences, and dependencies within the data.

SUPERVISED VS UNSUPERVISED (RECAP LINK)

Feature	Supervised	Unsupervised
Output Given?	✓ Yes	✗ No
Used For	Prediction	Pattern Discovery
Example	Churn, Fraud	Segmentation, Grouping
Output Type	Yes/No, Value	Clusters, Patterns

WHY BUSINESSES USE UNSUPERVISED LEARNING

Industry	Use Case
Banking	Suspicious transaction clustering
Marketing	Customer segmentation
Retail	Product bundling
HR	Employee behavior grouping
Healthcare	Disease subtype detection
Cyber Security	Anomaly detection

K-MEANS CLUSTERING (ADVANCED)

SESSION 1



Vector Math for Machine Learning

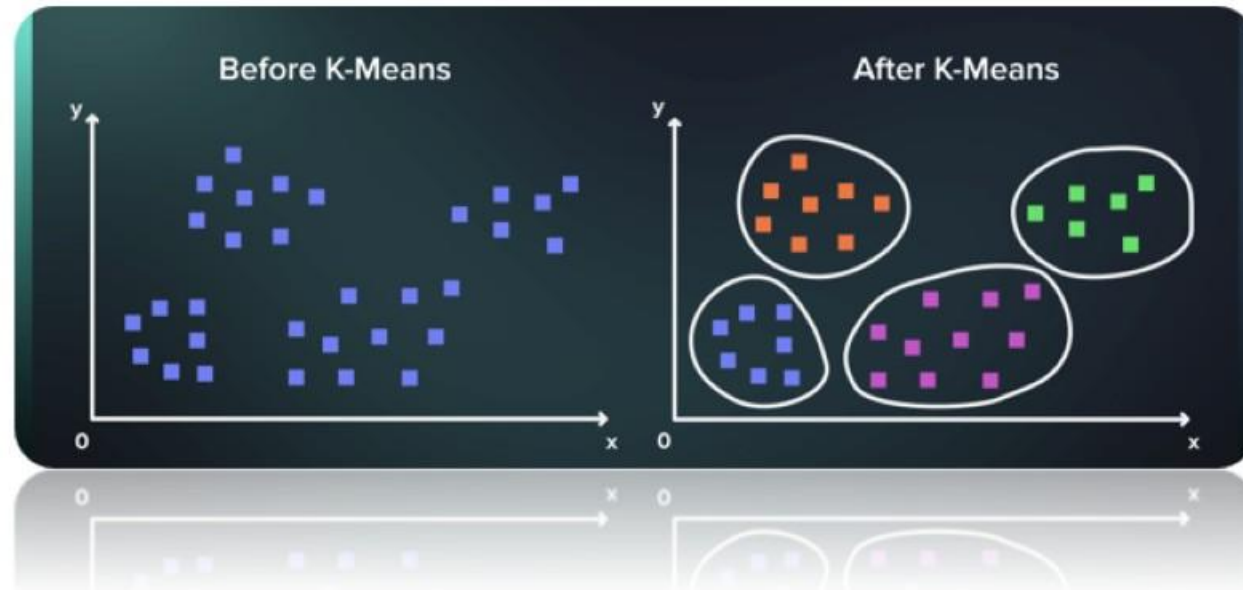
Operation	Meaning	Why Important in ML
Dot Product	Weighted similarity between vectors	Used inside every neuron, linear models
Euclidean Distance	Straight-line distance	KNN ((k-nearest neighbors) , clustering (K-means) , anomaly detection
Cosine Similarity	Direction similarity Values range: 1 → exactly similar 0 → no similarity -1 → opposite direction	NLP, recommendations, semantic search
Normalization	Scale vector to length 1	Feature scaling, stable training

WHAT IS CLUSTERING?

- **Clustering is the process of grouping similar data points together without knowing the output beforehand.**
- **Example**
 - Group similar customers
 - Group similar patients
 - Group similar employees

WHAT IS K-MEANS?

- K-Means is an unsupervised algorithm that divides data into K number of clusters where each data point belongs to the nearest cluster center.



HOW K-MEANS WORKS (STEP FLOW)

Choose number of clusters (K)

Randomly place centroids

Assign points to nearest centroid

Recalculate centroids

Repeat until stable

DISTANCE METRICS (REVISION + ADVANCED)

Metric	Use
Euclidean	Geometric distance
Manhattan	Grid-based
Minkowski	Generalized version

LAB 1A — ADVANCED CUSTOMER SEGMENTATION (RETAIL)

- **Business Case**
- Segment 200+ customers based on:
 - Income
 - Spending
 - Frequency
 - Discounts used
- **Objective**
 - Discover **VIP, Regular, Occasional and Risk customers** automatically.
- **Business Goal**
 - Retail companies want to **segment customers** to personalize:
 - Discounts
 - Loyalty offers
 - Upselling
 - Retargeting
 - Churn reduction
- We will cluster customers using:
 - **Income**
 - **Spending Score**
 - **Visit Frequency**
 - **Discount Usage %**



DAY 8 — UNSUPERVISED LEARNING (ADVANCED)

Learning without labels | Hidden patterns | Segmentation | Anomaly Detection

Recap

- **Day 6 — Tree-Based ML Models (Supervised Learning)**

- **1. Decision Trees**

- How splits are created (Gini / Entropy)
- Tree depth, branches, leaves
- Business interpretation using tree visualization
- Labs:
 - **1A** Loan Approval Decision System
 - **1B** HR Promotion Eligibility System
 - **1C** Insurance High-Risk Classification

- **2. Random Forest (Ensemble Learning)**

- Why ensembles work (reduce variance, avoid overfitting)
- Bagging technique
- Feature importance
- Business relevance in BFSI, Healthcare

- Labs:

- **2A** Fraud / High-Risk Patient Detection
- **2B** Health Risk Prediction (w/ suggestions based on medical ranges)
- **2C** Customer Loan Default Prediction (NBFC/Lending)

- **3. Model Comparison & Explainability**

- Compared models: Logistic vs Tree vs Forest
- Evaluation metrics: Accuracy, Precision, Recall, F1
- When to choose which model
- SHAP for explainability (feature contribution)
- Labs:
 - **3A** Multi-Model Comparison (Health Risk)
 - **3B** Churn Prediction Model Selection
 - **3C** Credit Risk Model Selection (BFSI)

Recap

-  **Day 7 — Unsupervised Learning (Advanced)**
- **Session 1 — Clustering & Association Analysis**
 - **1. Advanced Segmentation (K-Means)**
 - Unsupervised learning concept
 - Need for scaling (StandardScaler)
 - Elbow Method (k selection)
 - Labs:
 - **Lab 1A** Advanced Customer Segmentation (Income, Spending, Frequency, Discounts)
- **2. LAB 1B - Market Basket Analysis (Apriori)**
 - Transaction encoding
 - Frequent itemsets
 - Support, Confidence, Lift
 - Labs:
 - **Lab 1B** Retail Market Basket Analysis (Apriori)

DONE: 🔥 LAB 1A — ADVANCED CUSTOMER SEGMENTATION (RETAIL)

- **Business Case**
- Segment 200+ customers based on:
 - Income
 - Spending
 - Frequency
 - Discounts used
- **Objective**
 - Discover **VIP, Regular, Occasional and Risk customers** automatically.
- **Business Goal**
 - Retail companies want to **segment customers** to personalize:
 - Discounts
 - Loyalty offers
 - Upselling
 - Retargeting
 - Churn reduction
- We will cluster customers using:
 - **Income**
 - **Spending Score**
 - **Visit Frequency**
 - **Discount Usage %**

Learning from LAB 1A

-  **Why This Lab Is Important**
 - Retail segmentation improves marketing efficiency
 - Prevents wasteful discounts
 - Helps identify high-LTV customers
 - Enables personalized offers
- What we had practiced
 - How to generate & prepare customer data
 - StandardScaler + KMeans clustering
 - How to convert clusters into business labels
 - Visualizing customer groups

LAB 1B — Market Basket Analysis (Retail / E-Commerce)

- **Business Case**

- Retail stores track which items customers buy together.

- **Goal:**

- Identify **frequently purchased item combinations**
- Improve **store layout, cross-selling, bundle pricing, and discount strategy**

- **Examples:**

- Customers who buy *bread* often buy *butter*
- Customers who buy *shampoo* buy *conditioner*

- This is done using **Association Rule Mining** (Apriori).

- `pip install mlxtend`
- `from mlxtend.frequent_patterns import apriori, association_rules`
- `from mlxtend.preprocessing import TransactionEncoder    #For Encoded Dataframe`

LAB 1B — Market Basket Analysis (Retail / E-Commerce)

- **Why Market Basket Analysis?**

- Helps **cross-sell** (Amazon: “Customers also bought...”)
- Improves **store planning**
- Optimizes **bundle pricing, offers, coupons**

- **Key Concepts**

Term	Meaning
Support	How frequently an item appears in transactions
Confidence	Chance of buying B when A is bought
Lift	Strength of relation between A $\rightarrow$ B

- **What This Lab Demonstrates**

- Transaction encoding
- Apriori algorithm
- Rule generation
- Business insights from rules

- **Insights Like**

- Customers who buy **Beer** often buy **Chips**
- Customers buying **Diapers** tend to buy **Chips + Soda**

■ LAB 1C — Advanced Customer Lifetime Value (CLV) Segmentation (Retail / E-Commerce)

- **Business Case**

- A large e-commerce company wants to do **dynamic customer segmentation** based on:

- Monetary Value (Total Spend in last 12 months)
 - Frequency (No. of purchases/year)
 - Recency (Days since last purchase)
 - Returns Behavior (return % of orders)
 - Discount Sensitivity
 - Customer Tenure (years)
 - Traditional K-Means fails because the data has **non-linear patterns, outliers, and irregular density.**

- So the business wants a **hybrid segmentation system**:

- Step 1 → Normalize and prepare data
 - Step 2 → Identify base clusters using **K-Means**
 - Step 3 → Detect unusual customer patterns using **DBSCAN**
 - Step 4 → Assign CLV category (Platinum, Gold, Silver, At-Risk)
 - Step 5 → Provide actionable recommendations

- ■ **Objective**

- Build an **advanced multi-model segmentation system** and produce a final business decision table

LAB 1C — Advanced Customer Lifetime Value (CLV) Segmentation (Retail / E-Commerce)

- **1. Why this lab is advanced**
- This lab combines:
 - RFM-style analysis
 - K-Means for segmentation
 - DBSCAN for anomaly detection
 - A rule-based CLV model
 - Business recommendation engine
 - This mimics real-world e-commerce analytics.
- **2. What K-Means Does Here**
 - Finds natural customer groups using spending, frequency, discounts
 - Good for general segmentation
 - But fails with outliers → hence DBSCAN
- **3. What DBSCAN Adds**
 - Detects “weird patterns” such as:
 - High spend but low frequency
 - High returns
 - Very old or very new customers
 - Marks them as **Outlier customers**, which the business must handle separately
- **4. CLV Category Layer**
 - This is a **business mapping**, not ML. Using rules makes business interpretation clear.
- **5. Final Recommendation Layer**
 - This converts ML output → **action**. Without this, ML segmentation has no business value.

HOW TO DECIDE K VALUE (ADVANCED)

- ✓ Elbow Method
- ✓ Silhouette Score
- ✓ Business logic

Elbow Method – “Where the gain slows down”

- **Meaning**

- We calculate **Within-Cluster-Sum-of-Squares (WCSS)** for $K = 1$ to 10. WCSS tells us **how far points are from their cluster center**.
- As K increases $\rightarrow$ WCSS decreases.
- But after a certain K , the improvement becomes very small. This turning point looks like an **elbow**.
- **Choose the K value where adding more clusters doesn't significantly reduce the error.**

- **Visual example:**

- $K = 1$ $\rightarrow$ huge error
- $K = 2$ $\rightarrow$ major drop
- $K = 3$ $\rightarrow$ good improvement
- $K = 4$ $\rightarrow$ small improvement $\leftarrow$ ELBOW
- $K = 5+$ $\rightarrow$ almost flat line

- **Business interpretation:**

- *3 or 4 segments* are usually meaningful in Banking, Retail, Telecom.
- Too many segments confuse marketing teams.

Silhouette Score – “How well customers fit their cluster”

- Silhouette score measures how similar a customer is to:
 - its own cluster (cohesion)
 - other clusters (separation)
- Score range: **-1 to +1**
 - **+1** → Very good clustering
 - **0** → Overlapping clusters
 - **Negative** → Wrong cluster assignment
- "Higher silhouette score means the clusters are clear and well-separated."
- **Business interpretation:**
 - If K=4 gives silhouette 0.62 and K=5 gives 0.48 → **choose K=4**
 - Choose the K that gives the **highest score**, but also make sure it aligns with customer segments used by business.

Business Logic – “Does the segmentation make sense?”

- Even if metrics suggest $K=7$, the business may not need 7 segments.
- For example:
- **Retail example:**
 - VIP customers
 - High-value regular customers
 - Discount seekers
 - Inactive customers
- These **4 segments** are enough for marketing.
- **Banking example:**
 - Prime customers
 - Mass market
 - High-risk customers
 - Low engagement customers
- **Telecom example:**
 - Heavy users
 - Moderate users
 - Low users
 - Churn-prone users

HOW TO DECIDE K VALUE (ADVANCED)

-  Elbow Method
-  Silhouette Score
-  Business logic



Use the Elbow Method and Silhouette Score to find technically valid K values — then choose the one that aligns with business strategy.

◆ LAB 1D — Very Advanced: Mixed Clustering + PCA Visualization

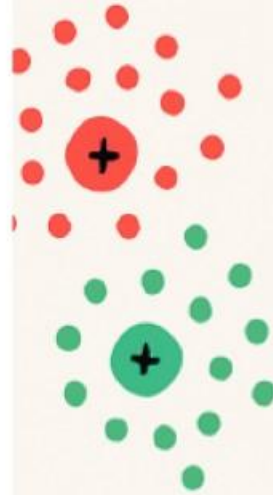
-  **Objective**
 - Use multiple clustering techniques to discover customer segments, compare cluster quality, visualize clusters in 2D using PCA, and recommend which segmentation a retail business should use and why.
 - Report **silhouette score** for each method and comment on stability/interpretability.
 - Provide final recommended segmentation (method + business naming) and action plan for each segment (2–3 actions).
 - (Optional) Try GaussianMixture or tune DBSCAN parameters and submit result
- **Business context**
 - E-commerce wants a robust segmentation for targeted marketing and risk management. Different clustering methods may find different structures — your job is to test several, visualize results, compare them quantitatively, and give business recommendations.
- **Deliverables**
 - Cluster customers using **KMeans**, **Agglomerative (hierarchical)**, and **DBSCAN**.
 - Reduce features to 2D using **PCA** and produce scatter plots for each method (clusters color-coded).
- **IMPORT**

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans,
AgglomerativeClustering, DBSCAN
from sklearn.metrics import silhouette_score
from sklearn.neighbors import NearestNeighbors
```

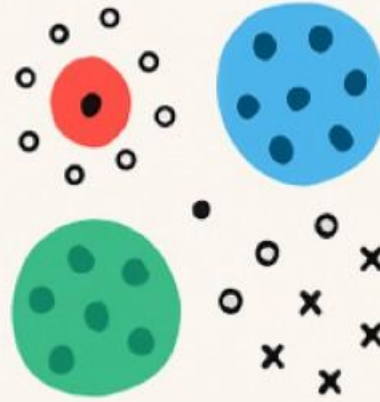
◆ LAB 1D — Very Advanced: Mixed Clustering + PCA Visualization

- **Why multiple algorithms?**
 - KMeans finds compact spherical clusters.
 - Agglomerative shows hierarchical relationships.
 - DBSCAN finds dense clusters and flags noise/outliers.
 - Use all three to understand data structure.
- **Why PCA?**
 - PCA is only for visualization. Clusters in PCA plane are an approximation; always verify via metrics (silhouette).
- **Silhouette score**
 - is a good numeric guide (range -1..1). Higher is better. But interpret alongside business sense.
- **DBSCAN tuning:**
 - show k-distance plot; ask participants to check eps values and min_samples.
- **Naming clusters:**
 - Use profiling (mean of features) to give business-friendly names — not purely automatic labels.
- **Business recommendation template (Tasks)**
- **For each final segment include:**
 - Segment name
 - % of customers in segment
 - 2 marketing actions (campaign, discount strategy)
 - Expected business outcome (increase ARPU, reduce Churn)

K-means



DBSCAN



- Core point
- Border point
- × Noise
- × Noise
- Border point
- Core point

**Hierarchical
Clustering**



HIERARCHICAL CLUSTERING & DBSCAN

Session 2

LIMITATIONS OF K-MEANS

- ✗ Must choose K in advance
- ✗ Sensitive to noise
- ✗ Fails with irregular shapes

HIERARCHICAL CLUSTERING

Hierarchical clustering builds a tree of clusters that shows how data merges step-by-step.

- ✓ No need to choose K initially
- ✓ Gives full dendrogram

HIERARCHICAL CLUSTERING

- Hierarchical clustering is an **unsupervised learning technique** that builds clusters in a **tree-like structure (dendrogram)**.
 - It does NOT require selecting K at the beginning.
 - Instead, clusters are formed step-by-step until all points merge.
- **How it works (Simple Explanation)**
 - Hierarchical clustering works in two general ways, but we use **Agglomerative** (bottom-up):
 - **Agglomerative (Bottom-Up)**
 - Every customer starts as a separate cluster
 - The model finds the **two closest customers/clusters**
 - It merges them
 - Repeats until only one cluster remains
 - We cut the dendrogram at a chosen height to decide the K clusters

HIERARCHICAL CLUSTERING

- **Distance Measures**

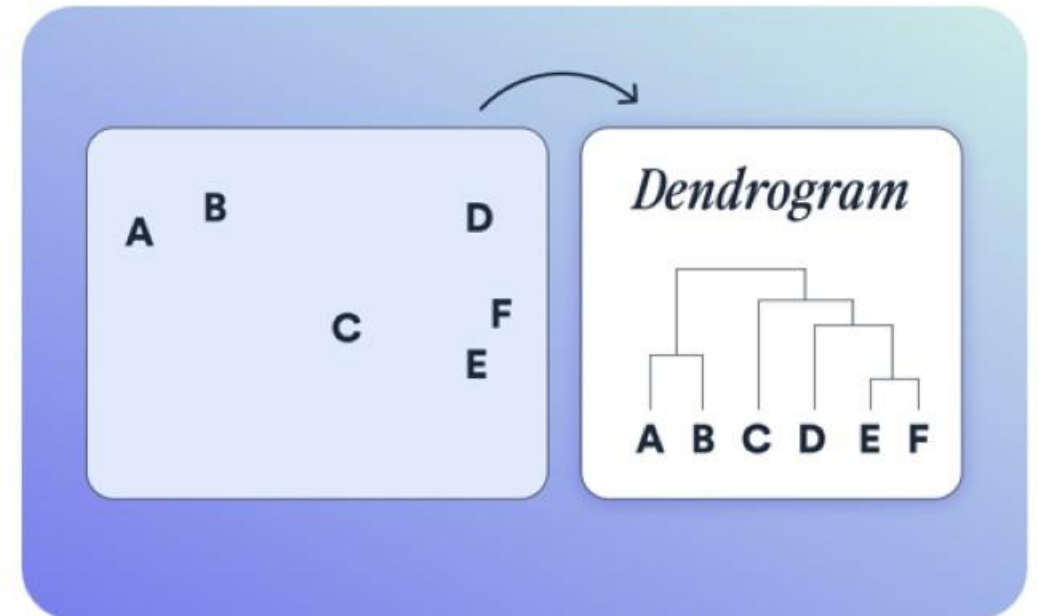
- Hierarchical clustering requires a distance measure, such as:
 - Euclidean
 - Manhattan
 - Cosine

- **Linkage Methods**

- Linkage decides how the distance between clusters is measured:
 - **Single Linkage** – minimum distance
 - **Complete Linkage** – maximum distance
 - **Average Linkage** – average distance
 - **Ward's Linkage (most used)** – minimizes variance within clusters

DENDROGRAM EXPLAINED

- A dendrogram visual shows:
 - Where clusters merge
 - Distance of each merge
 - Natural segmentation
- Each vertical cut = new cluster
- Higher cut = fewer clusters
- Lower cut = more clusters
- **Where it is used (Business Use Cases)**
 - Customer segmentation (Retail & Banking)
 - Fraud pattern clustering
 - Patient similarity grouping (Healthcare)
 - Document similarity
 - Market basket behavior



DENDROGRAM EXPLAINED

- **Advantages**

- No need to predefine K
- Very interpretable using dendrogram
- Works well for smaller datasets (up to 5000–10,000 rows)
- Good for exploratory analysis

- **Disadvantages**

- Slow for very large datasets
- Sensitive to scaling
- Hard to handle noisy data or outliers

LAB 2A — EMPLOYEE PERFORMANCE CLUSTERING (HR)

- **Business Case**

- Cluster employees based on:

- Productivity
- Attendance
- Learning speed
- Collaboration score

- **Outcome**

- Identify:

- High Potential Talent

- Average Workers

- At Risk Group

- **Questions –**

- Choose k based on dendrogram + silhouette score + business sense. For HR, 3 clusters (High Potential, Average, At Risk) are common and actionable.

LAB 2A — EMPLOYEE PERFORMANCE CLUSTERING (HR)

-  **Recommended feature ranges**
 - **Productivity:** 0 – 100 (higher = better)
 - **Attendance (%):** 0 – 100 (higher = better)
 - **LearningSpeed:** 1 – 5 (1 = slow learner, 5 = fast learner)
 - **CollaborationScore:** 1 – 5 (1 = poor, 5 = excellent)
- These ranges are business-friendly and easy for HR to understand. You can adjust scales to match your organization.

LAB 2A — EMPLOYEE PERFORMANCE CLUSTERING (HR)

- **4. Centroid-based assignment for new employees.**
AgglomerativeClustering cannot predict — nearest-centroid in scaled space is a reliable approach for assigning new employees to existing clusters.
- **5. Business actions** for each segment and how HR teams can operationalize recommendations (L&D programs, PIPs, succession planning).
- **6. Experimentation.** Let try different linkage methods (single, complete, average, ward) and different cluster counts to see effects.
-

2. DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

2. DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

- DBSCAN is a density-based clustering algorithm that **groups together points that are close and dense** and marks points in low-density areas as **outliers**.

Unlike K-Means or Hierarchical Clustering:

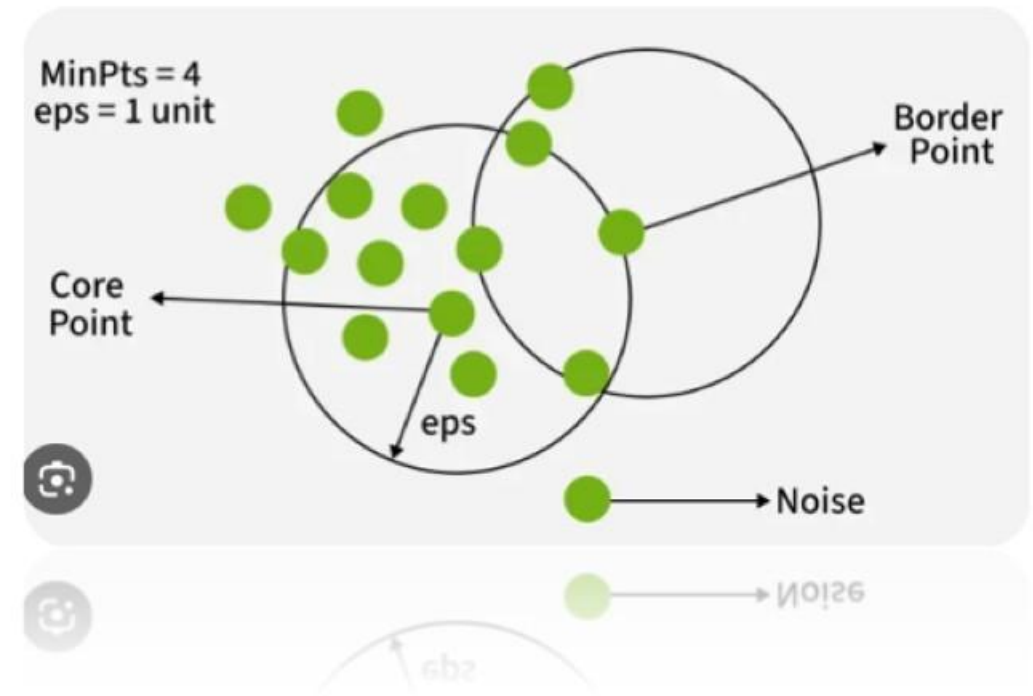
- Does NOT require K
- Automatically finds clusters of different shapes
- Automatically detects outliers/anomalies

2. DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

- **How it works**
- DBSCAN needs two parameters:
 - **1. eps (ϵ)**
 - The radius within which the algorithm looks for neighbors.
- **2. min_samples**
 - Minimum number of points required inside eps to form a dense region (cluster).

2. DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

- **Types of Points in DBSCAN**
- DBSCAN categorizes points into:
 - **Core Points**
Points with at least *min_samples* neighbors inside the *eps* radius.
 - **Border Points**
Points inside a cluster but with fewer neighbors than required to be core.
 - **Noise Points (Outliers)**
Points that do not belong to any cluster.



DBSCAN (DENSITY BASED CLUSTERING)

- **DBSCAN forms clusters based on data density and automatically detects outliers.**
 -  No need to choose K
 -  Detects fraud-like patterns
 -  Powerful for anomaly detection
- **Why DBSCAN Is Useful**
 - Finds clusters of **arbitrary shapes** (K-Means only finds circular clusters)
 - Automatically identifies **outliers**
 - Works well where cluster sizes are uneven

DBSCAN (DENSITY BASED CLUSTERING)

- **Where DBSCAN is used (Business Use Cases)**

- **Banking**

- Fraud detection
- Unusual transaction behavior
- Anomaly detection in credit card usage

- **Retail**

- Detecting rare customer segments
- Outlier spenders

- Locating clusters of stores by geography

- **Healthcare**

- Detecting abnormal patient vitals
- Disease outbreak hotspots

- **Telecom**

- Network anomaly detection
- Unusual usage patterns

DBSCAN (DENSITY BASED CLUSTERING)

- **Advantages**

- No need for K
- Automatically detects outliers
- Works well with uneven cluster shapes
- Highly effective for anomaly and fraud detection

- **Disadvantages**

- Choosing eps and min_samples can be tricky
- Struggles with varying density levels
- Sensitive to scaling (must normalize data)

Comparison Summary

Feature	Hierarchical	DBSCAN
Need K?	No, optional	No
Handles Outliers	Poor	Excellent
Shape of Clusters	Mostly spherical	Any shape
Interpretation	Very good (dendrogram)	Medium
Suitable Size	Small/Medium	Medium/Large
Best For	Segmentation	Anomaly/Fraud detection

LAB 2B — FRAUD ANOMALY DETECTION USING DBSCAN

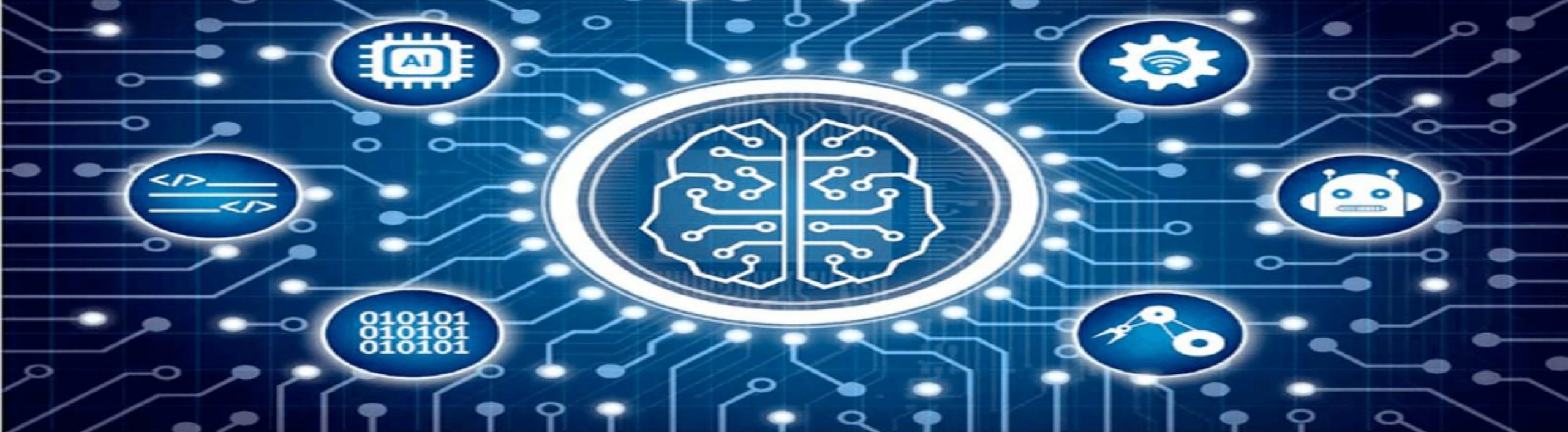
- **Business Case (Banking / NBFC / FinTech)**
 - Fraud analysts want to detect **unusual transactions** such as:
 - Rare **high-value spikes**
 - **Sleeper accounts** suddenly showing activity
 - **Multiple transactions grouped in a short time**
 - **Unusual spending patterns**
- **DBSCAN is ideal because:**
 - It finds **dense normal groups**
 - Flags **isolated abnormal points**
 - Does NOT require specifying number of clusters
 - Detects **noise/fraud points automatically**

LAB 2B — FRAUD ANOMALY DETECTION USING DBSCAN

- **CHECK-**
 - Unexpected behavior compared to past
- **A) Normal customers (Cluster 0,1,2)**
 - Stable, predictable behavior. Safe.
- **B) Fraud / Anomaly customers (Label = -1)**
 - These need investigation:
 - Sudden INR 20,000+ debit
 - Frequency spike (3 → 15 per day)
- **C) Sleeper accounts**
 - Low amount but sudden burst → high red flag for mule account.

Additional Labs

- **✓ Lab 2C — LENDING DEFAULT ANOMALY DETECTION (DBSCAN + Features)**
- **✓ Lab 2D — CREDIT CARD FRAUD (Multi-feature anomaly)**



Artificial Intelligence & Machine Learning

Phase 3

With,

Dr. Swati Verma

L&D Consultant, Master Trainer, Capacity Enhancement Coach

DAY 9 –



DIMENSIONALITY REDUCTION & ANOMALY DETECTION

SESSION 3

INTRODUCTION — DIMENSIONALITY REDUCTION & ANOMALY DETECTION

- Why ?
- In real business datasets (banking, retail, telecom, healthcare), we often have:
 - 20, 50, or 100+ features
 - Highly correlated attributes
 - Noise in data
 - High computation time
- This causes:
 - Overfitting
 - Slow training
 - Difficult visualization
 - Harder anomaly detection
- **Dimensionality Reduction = Simplifying the dataset while preserving information.**

WHY DIMENSIONALITY REDUCTION?

Problem	Solution
Too many columns	PCA
Slow computation	PCA
Visualization required	PCA

PCA (PRINCIPAL COMPONENT ANALYSIS)

- A **dimensionality reduction technique**
- Converts many features → few uncorrelated components
- Retains **maximum variance (information)**
- Helps visualize complex data
- Used before clustering, anomaly detection, segmentation, churn modelling

- **PCA reduces the number of features while keeping maximum information.**
 -  Data compression
 -  Noise reduction
 -  Visualization in 2D

Two Popular Techniques

- **A. PCA (Principal Component Analysis)**

- Used when:
 - You want to **compress data**
 - You want a **2D/3D visual view of clusters**
 - You deal with **highly correlated variables**
- PCA:
 - Creates *new variables* called **Principal Components**
 - Removes noise
 - Keeps maximum variance
 - Excellent for visualization + anomaly detection
-

- **B. t-SNE (t-Distributed Stochastic Neighbor Embedding)**

- Used when:
 - Dataset is complex & non-linear
- You want **visual clusters** for:
 - Images
 - High-dimensional customer behavior
 - Embedding vectors
- You need human-friendly patterns
- Not used for model training — only visualization.

Why use Dimensionality Reduction before Anomaly Detection?

- **Why use Dimensionality Reduction before Anomaly Detection?**
- Because anomalies become more obvious when:
 - Noise is removed
 - Data is compressed into 2D
 - Patterns visually separate out
- Example:
 - In PCA plot, most credit transactions cluster tightly
 - Fraud transactions appear far away → easy to flag

Techniques for Anomaly Detection

- **Isolation Forest** → isolates outliers
 - **Local Outlier Factor (LOF)** → density-based
 - **DBSCAN** → cluster + anomaly points
 - **Autoencoder (Deep Learning)** → reconstruction error
 - For today, we'll use:
- **PCA (to visualize)**
 - **Isolation Forest (to detect anomalies)**

Why do we need PCA?

Problem	PCA Benefit
Customer data is high-dimensional	Reduces complexity
Hard to visualize 6–10 metrics	Converts to 2D/3D plotting
Features might be correlated	PCA removes multicollinearity
Want a single score for “value” or “engagement”	PCA gives composite factors

Real Use Cases

- Identify **VIP** vs **Low-value** customers
- Detect **behavior drift** (COVID impact etc.)
- Segment for marketing campaigns
- Visualize **customer journey patterns**



LAB 3A — CUSTOMER BEHAVIOR PCA VISUALIZATION

- **Business Context (Retail / E-commerce / Banking)**
- Organizations collect many behavioural signals:
 - Annual income
 - Spending score
 - Visit frequency
 - Complaints
 - Discounts used
 - Loyalty score
 - Tenure
- The challenge: **Too many features → difficult to visualize customer patterns.**
- **PCA (Principal Component Analysis)** solves this by converting all features into **2 or 3** **meaningful business dimensions** such as:
 - Component 1 → "Buying Power"
 - Component 2 → "Engagement Level"
 - This enables powerful **segmentation, drift analysis, and clustering.**
 -
-  **Objective**
 - Reduce **6D customer data** to **2D** using PCA
 - Visualize natural behavioural patterns
 - Identify customer types using scatter plot
 - Interpret components as business features

ANOMALY DETECTION (BUSINESS CRITICAL)

- **Anomalies are rare unusual events that do not follow normal behavior patterns.**
- Examples:
 - Fraud
 - System attack
 - Fake clicks
 - Equipment failure

LAB 3B — CREDIT CARD ANOMALY DETECTION (PCA + Isolation Forest)

- **Business Case**

- A Bank wants to identify **suspicious credit card transactions** such as:
- Unusual spike in spend
- Transactions from abnormal locations
- Transactions beyond normal customer pattern

- **Goal:**

- Detect anomalies using **Isolation Forest**
- Visualize normal vs anomaly using

PCA (2D plot)

- Generate business-friendly interpretation
- **Imports:**
- from sklearn.ensemble import IsolationForest
- from sklearn.decomposition import PCA

LAB 3B — CREDIT CARD ANOMALY DETECTION (PCA + Isolation Forest)

- **1. Anomaly Detection Logic**

- Isolation Forest isolates rare patterns
- Fraud = datapoints that are “easy to isolate”

- **2. PCA Interpretation**

- Participants can clearly see:
 - Blue cluster = normal transactions
 - Red scattered points = fraudulent ones

- **3. User Input + Business Output**

- Participants love this because:
 - They enter transaction amount/time/distance
 - Model instantly tells “FAKE / REAL”
- This is real-world BFSI workflow.

Summary

Built **Decision Tree, Random Forest, Clustering, Market Basket** models

Understood **feature importance & explainability**

Compared models using **real business cases**

Practiced end-to-end ML workflow

Learned both **supervised and unsupervised** techniques



Day 9 - ADVANCED AI FOUNDATIONS

DEEP LEARNING + TENSORFLOW/KERAS + NLP

Deep Learning + NLP Foundations

Corporate AI Training

Technically intense —ANN + TensorFlow + NLP.

Agenda

- Deep Learning Fundamentals
- Neural Network Architecture
- Activation Functions + Loss
- TensorFlow/Keras Essentials
- ANN for Classification & Regression
- NLP Foundations
- NLP Business Use Cases

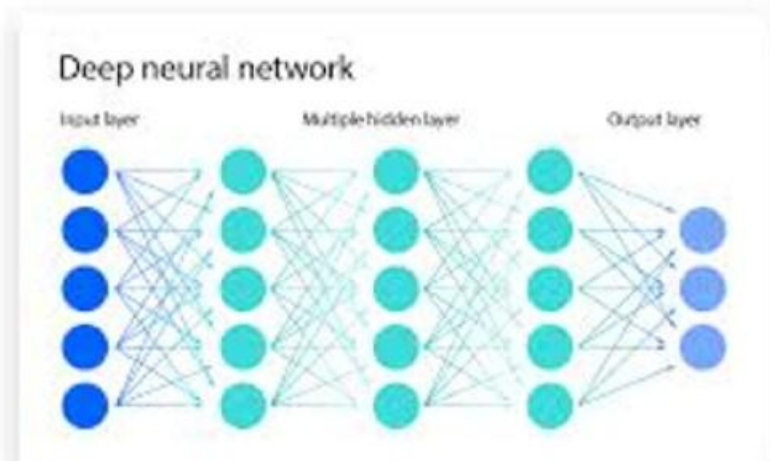
Recap of Previous Sessions



Tree-based ML models (Decision Trees, Random Forest)
Model Comparison & Bias–Variance
Unsupervised Learning
K-Means, Hierarchical, DBSCAN
PCA + Anomaly Detection

Why Deep Learning? (Business View)

- Deep Learning is used when:
 - Relationships are nonlinear
 - Feature space is high-dimensional
 - Traditional ML underperforms
 - Data volume is large
- Industry Examples:
 - Banking: Fraud detection, credit scoring
 - Retail: Customer churn, demand forecasting
 - Healthcare: Disease prediction
 - Manufacturing: Defect detection
 - HR: Attrition prediction



ANN Regression Introduction

- **Artificial Neural Network (ANN) regression** is a powerful machine learning technique that uses a neural network architecture to predict a **continuous numerical output** (e.g., house price, temperature, stock value) based on a set of input variables.
- It is particularly effective for modeling complex, non-linear relationships that traditional linear regression models may struggle to capture.

Core Concepts of ANN

- **Mimics the Human Brain:** ANNs are designed to loosely mimic the structure and function of the human brain, using interconnected artificial neurons arranged in layers to process information and learn patterns from data.
- **Architecture:** A typical ANN for regression consists of three main components:
 - **Input Layer:** Receives the raw input features (independent variables). The number of neurons in this layer matches the number of input features.
 - **Hidden Layers:** One or more layers that perform the majority of the processing, transforming the input data through weighted sums and activation functions to extract complex patterns.
 - **Output Layer:** The final layer that produces the model's prediction. For a regression problem, this layer typically has a single neuron with a linear activation function to output a continuous value.

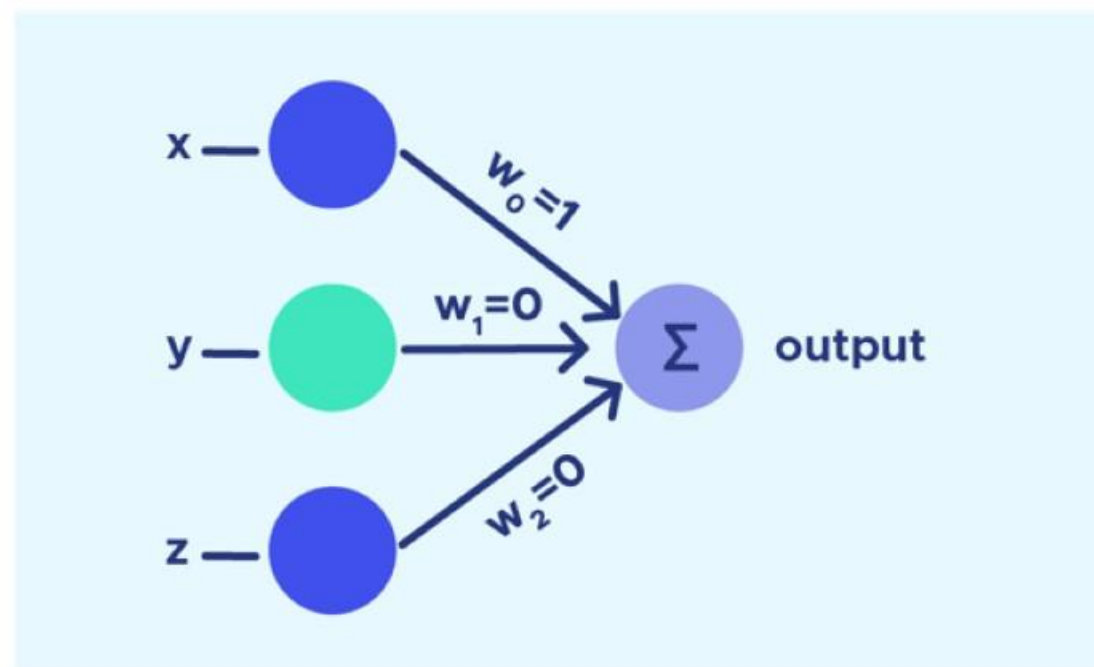
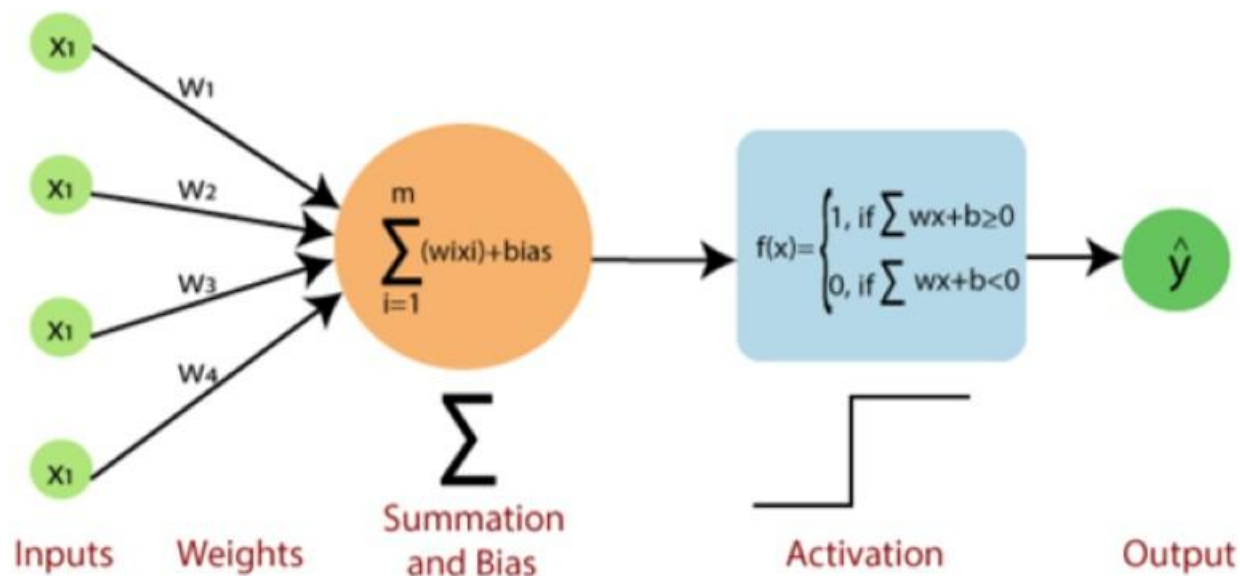
Core Concepts of ANN

- **Learning Process:** The network learns by adjusting the weights (importance) and biases of the connections between neurons during a training process.
- This is achieved using an optimization algorithm (like gradient descent) and a **loss function**, commonly the Mean Squared Error (MSE), to minimize the difference between the predicted values and the actual target values.
- The error is propagated backward through the network in a process called **backpropagation** to fine-tune the parameters.

The Artificial Neuron

- **Formula:**

$$\text{Output} = \text{Activation}(W_1X_1 + W_2X_2 + \dots + b)$$



Key Advantages

- **Non-linearity:** ANNs can model highly complex, non-linear relationships without needing the user to specify the exact functional form in advance.
- **Feature Learning:** They can automatically learn hierarchical features from the data, reducing the need for explicit feature engineering.
- **Scalability:** ANNs perform well with large amounts of data, often outperforming traditional machine learning algorithms as data size increases.

Applications of ANN

- ANN regression is used in various fields, including:
 - **Financial forecasting** (e.g., stock prices)
 - **House price prediction**
 - **Energy demand forecasting**
 - **Scientific data analysis** (e.g., predicting chemical properties)

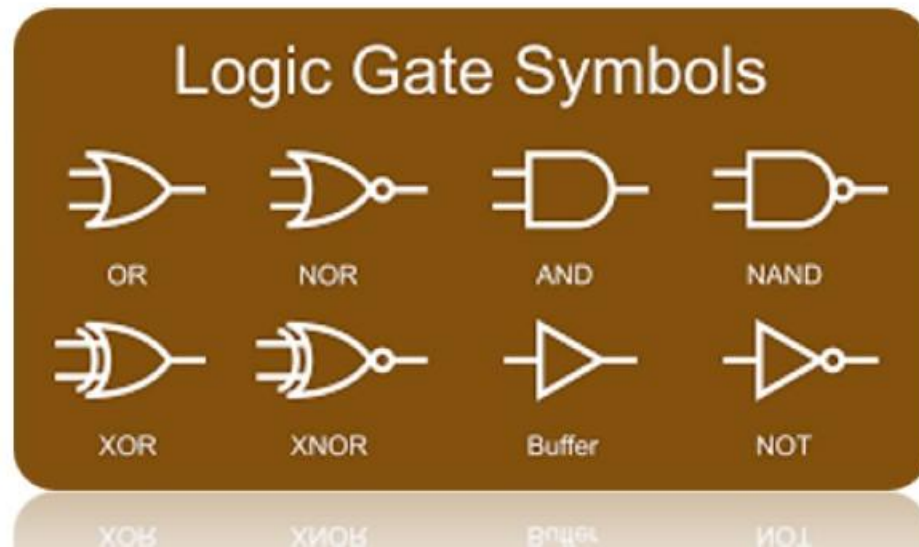
Implementation Steps of ANN

- Building an ANN regression model typically involves:
 - **Data Collection & Preprocessing:** Cleaning data, handling missing values, and normalizing or scaling features are essential steps.
 - **Model Definition:** Specifying the network architecture, including the number of layers, neurons per layer, and activation functions.
 - **Model Compilation:** Defining the loss function (e.g., MSE) and the optimizer (e.g., Adam).
 - **Training:** Fitting the model to the training data, adjusting weights iteratively over a number of epochs (full passes through the data).
 - **Evaluation:** Assessing the model's performance on a separate validation or test dataset using metrics like Mean Absolute Error (MAE) or Root Mean Squared Error (RMSE).
 - **Prediction:** Using the trained model to make predictions on new, unseen data.
- Frameworks like TensorFlow and Keras provide tools to easily implement ANN regression models.

SESSION 1 – DEEP LEARNING WITH ANN (Keras)

Boolean Gates $\rightarrow$ ANN Foundation

- AND gate
- OR gate
- NOT gate
- XOR gate (not linearly separable $\rightarrow$ need multi-layer NN)



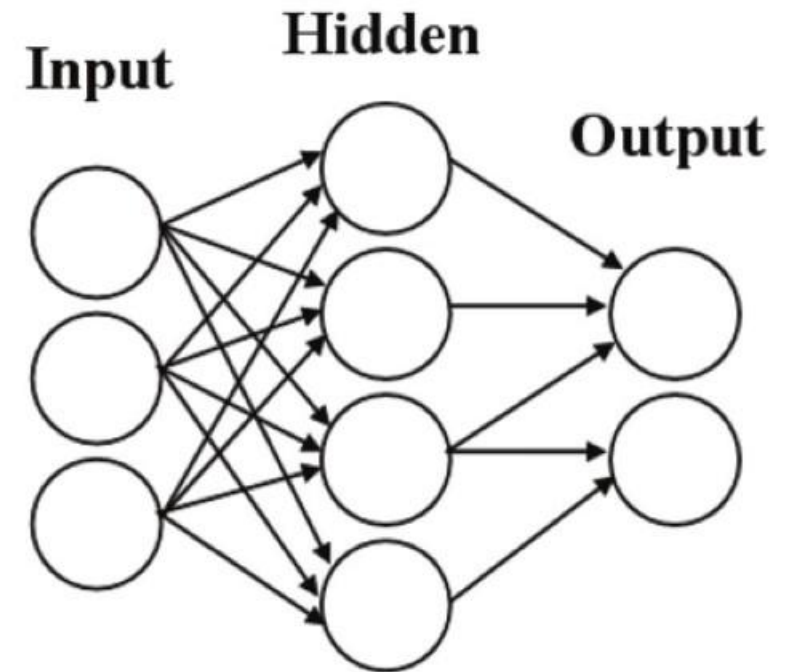
ANN Architecture

- **Layers:**

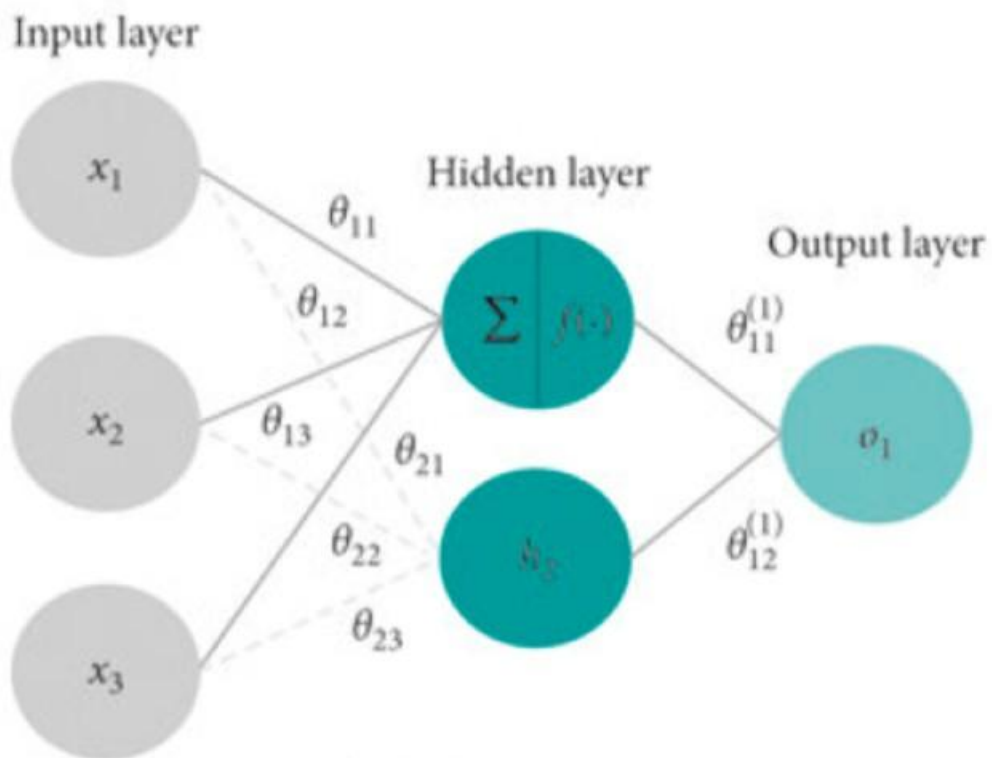
- Input layer
- Hidden layers
- Output layer

- **Flow:**

Input $\rightarrow$ Weighted Sum $\rightarrow$ Activation $\rightarrow$
Output



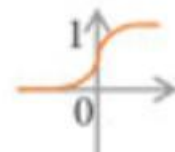
ANN Architecture



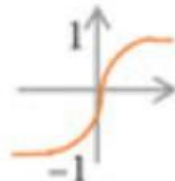
$$h_j = f(\sum_{i=1}^{\text{input units}} \theta_{ji} x_i)$$

$$o_1 = f(\sum_{i=1}^{\text{hidden units}} \theta_{1i}^{(1)} h_i)$$

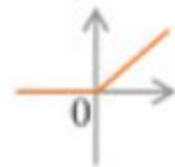
$$f(z) \xrightarrow{\text{sigmoid}} 1 / (1 + e^{-z})$$



$$f(z) \xrightarrow{\text{tanh}} (e^z - e^{-z}) / (e^z + e^{-z})$$



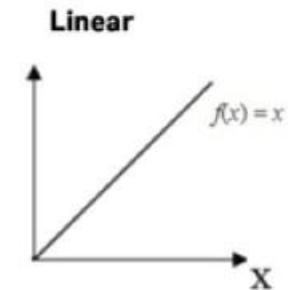
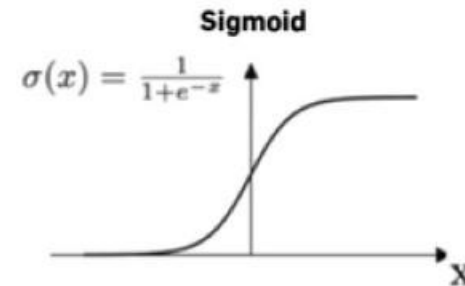
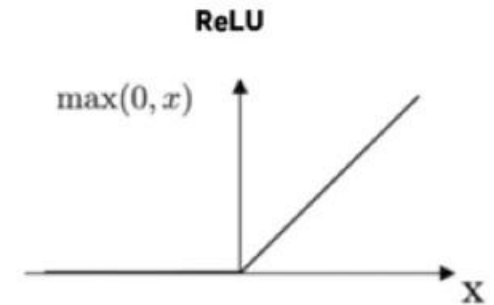
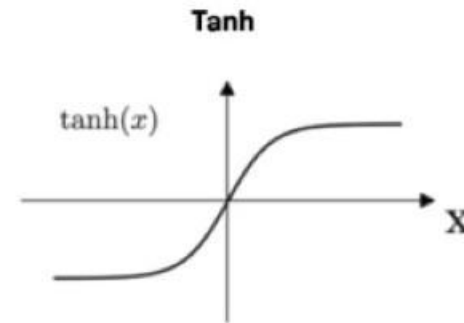
$$f(z) \xrightarrow{\text{relu}} \max(0, z)$$



Activation Functions

- **Why activation?**

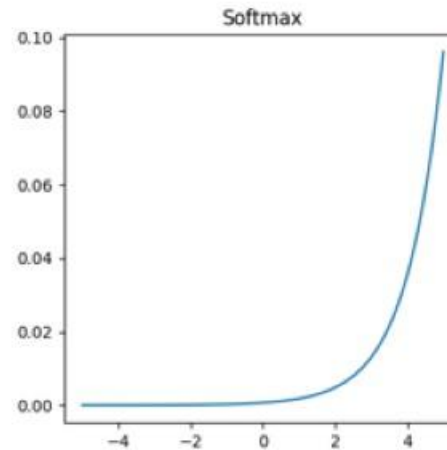
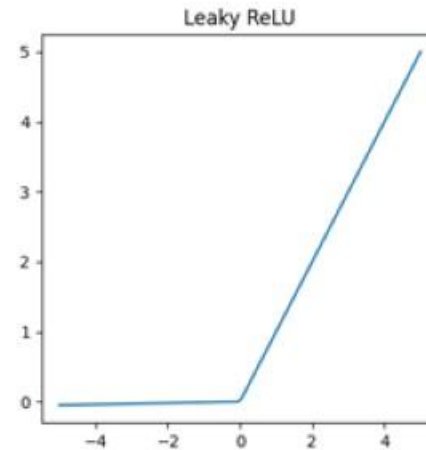
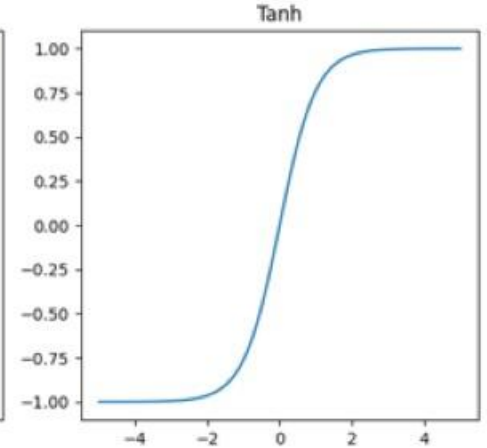
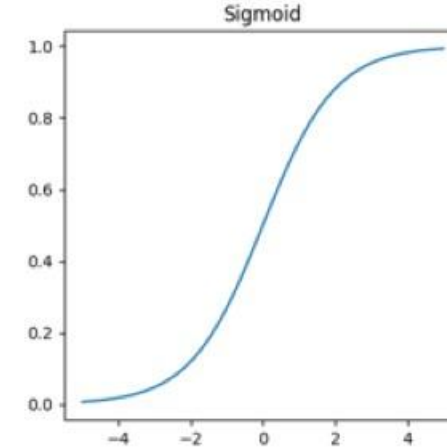
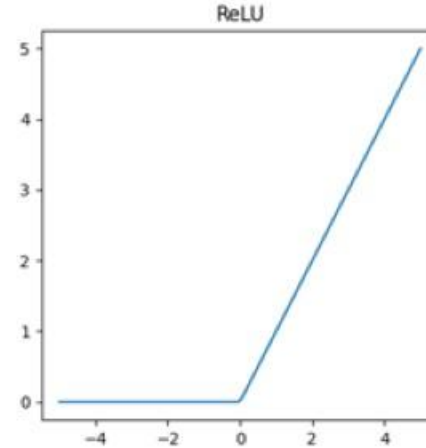
To add non-linearity $\rightarrow$ model can learn complex patterns.



Activation Functions

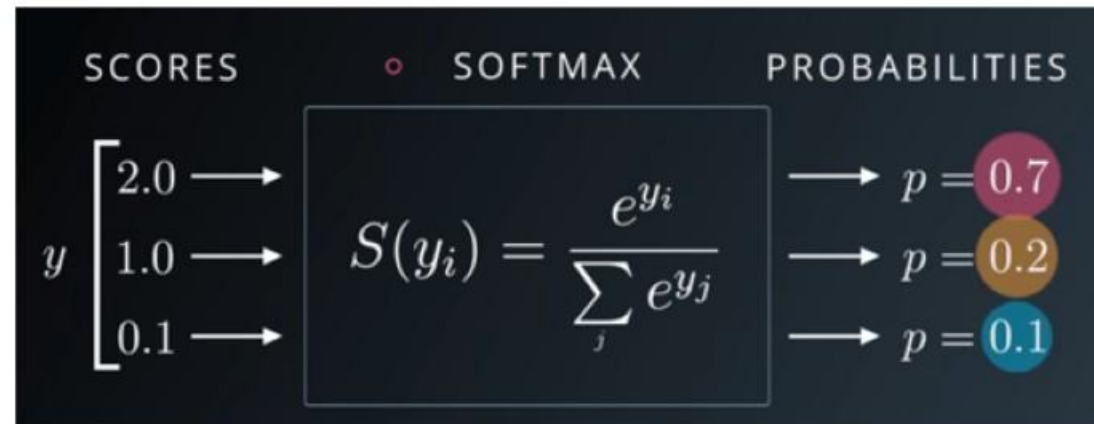
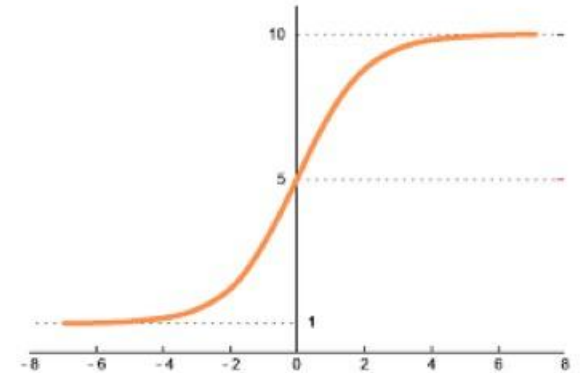
- **Common activations:**

- Sigmoid
- Tanh
- ReLU
- Leaky ReLU
- Softmax



Softmax

- Softmax converts raw scores into **probabilities that sum to 100%**.
- **Use case:**
 - Churn: Yes / No
 - Fraud risk: High / Medium / Low
 - Sentiment: Positive / Neutral / Negative



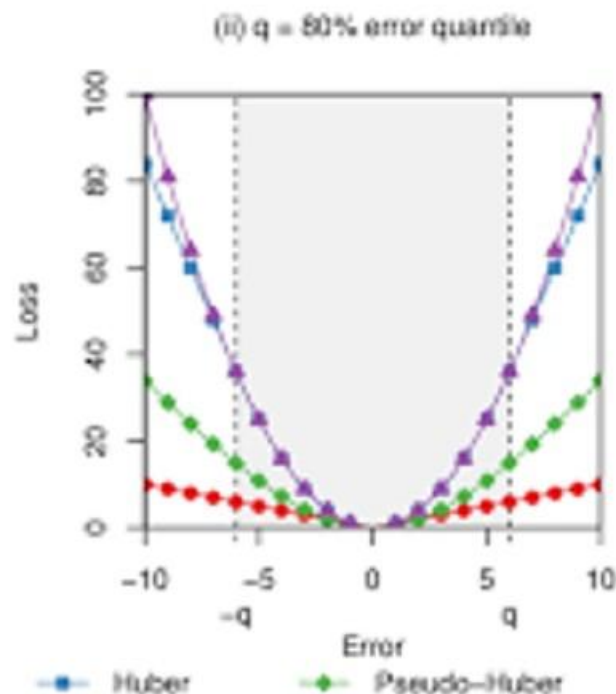
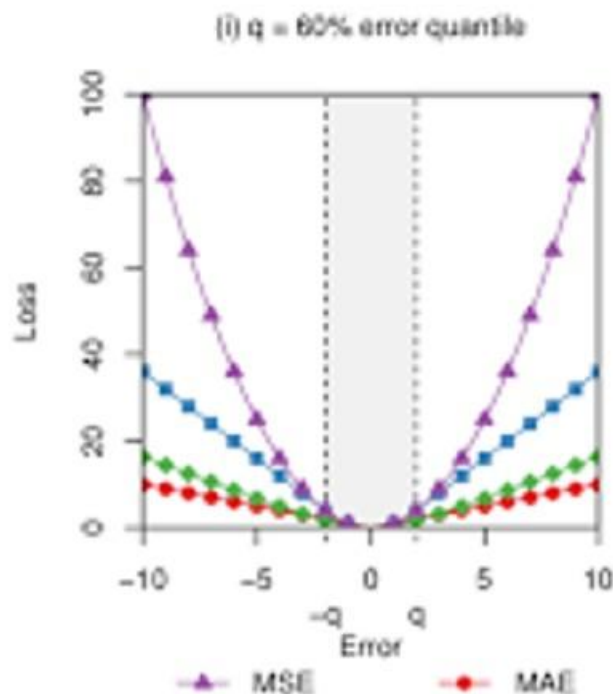
Loss Functions

- **Regression:**

- MSE - Mean Squared Error
- MAE - Mean Absolute Error

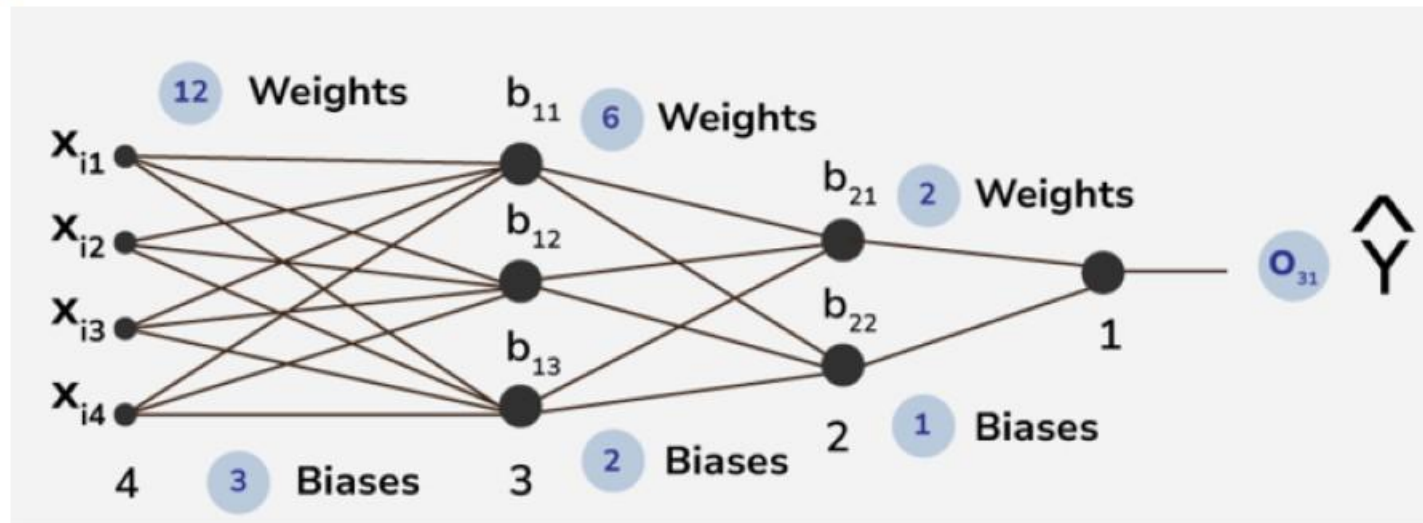
- **Classification:**

- Binary cross-entropy
- Categorical cross-entropy



Forward Propagation (Intuition)

- Input flows into network
- Weighted sums computed
- Activation applied
- Loss calculated
- Compare prediction vs actual



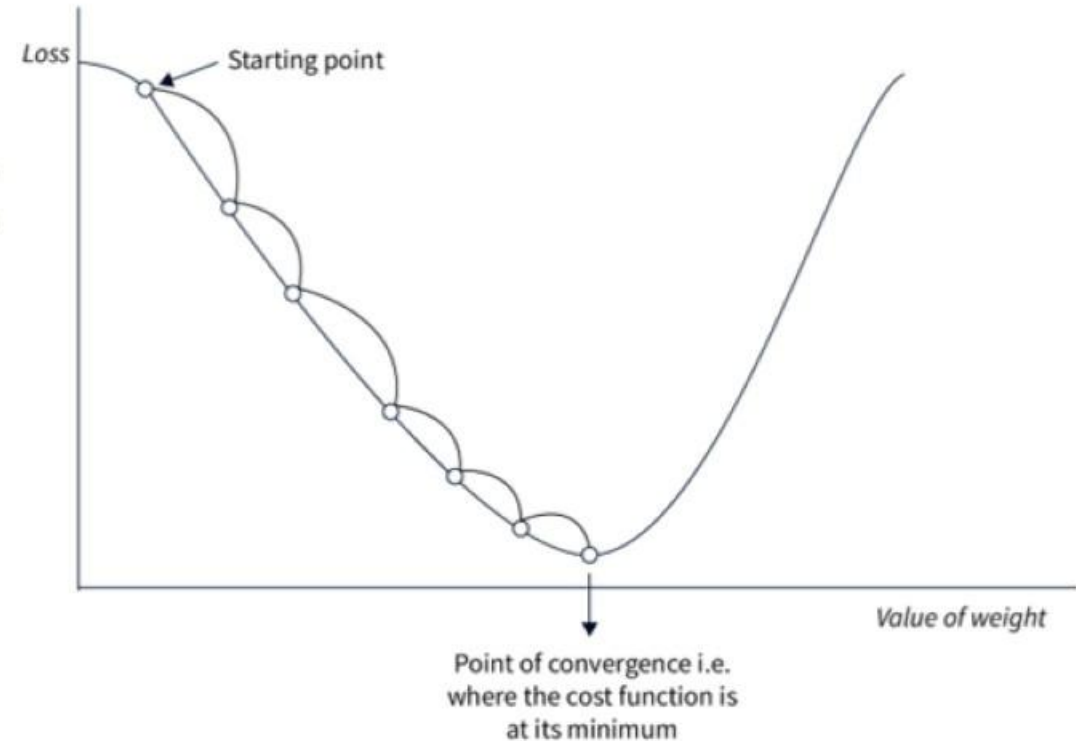
Optimizers

- **First-order optimizers:**

- Gradient Descent
- SGD (Stochastic Gradient Descent)
- Momentum

- **Adaptive optimizers:**

- RMSProp (Root Mean Square Propagation)
- Adam (Adaptive Moment Estimation) (industry standard)



Hyperparameters in ANN

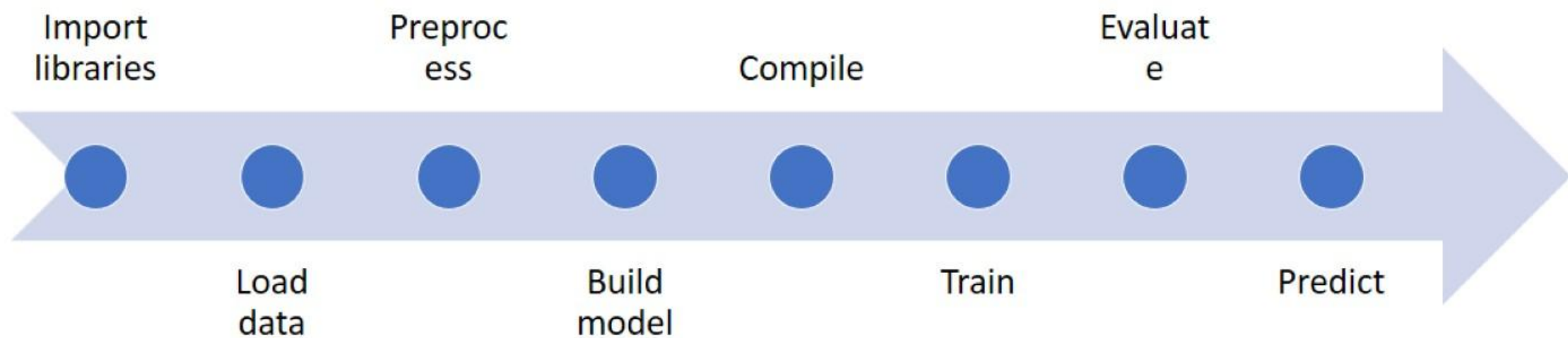
- **Tuning parameters:**

- Number of hidden layers
- Number of neurons
- Learning rate
- Batch size
- Epochs
- Dropout
- Early stopping

Introduction to TensorFlow & Keras

- **TensorFlow:** Deep Learning engine
- **Keras:** High-level API
 - Sequential model
 - Compile
 - Fit
 - Predict

Building an ANN (Keras Workflow)



LAB 1A — Insurance Claim Approval Prediction (ANN Classification)

- **Business Case**
- A Health Insurance company wants to predict whether a claim should be **approved/rejected** based on:
 - Age
 - Claim Amount
 - Medical Score
 - Number of Previous Claims
- Outcome helps underwriters reduce manual load.
- Import list:
- **Use PyTorch for:**
 - ANN basics
 - ANN Regression
 - ANN Classification
 - CNN overview/demos
- **Use sklearn for:**
 - NLP (TF-IDF + Logistic Regression)
 - Spam Classification
 - Sentiment Analysis
 - Quick ANN Demo (MLPClassifier)

LAB 1A — Insurance Claim Approval Prediction (ANN Classification)

- An Insurance Company wants to automate claim approvals using ANN.
They want to predict whether a claim should be:
 - **Approved (1)**
 - **Rejected (0)**
- Based on customer attributes:
 - Age
 - Claim Amount
 - Claim History Score
 - Policy Tenure
 - Fraud Risk Score
- **Objective:**
Build an ANN model using PyTorch (preferred) or sklearn (backup) to predict approval.

LAB 1A — Insurance Claim Approval Prediction (ANN Classification)

- **Learning Outcomes**

- ANN binary classification
- Sigmoid + BCE
- Real-world insurance use case

LAB 1B — Retail Demand Forecasting (ANN Regression)

- A retail chain wants to forecast **monthly product demand** using ANN based on:
 - **Marketing Spend** (in INR)
 - **Store Footfall** (number of customers)
 - **Seasonality Index** (0.5 = off-season, 1 = normal, 1.5 = festive)
 - **Competitor Activity Score** (0–100)
- **Goal**
 - Predict **Monthly Demand (units)** using a regression ANN.
- **Learning Outcomes**
 - ANN Regression
 - Linear output layer
 - Model training & evaluation (MSE/RMSE)
 - Business forecasting interpretation

LAB 1C: ANN Regression (House Price Prediction)

- **Objective:**

- Predict property prices using ANN regression.

- **Note:**

Use relu for hidden layers, linear for output.

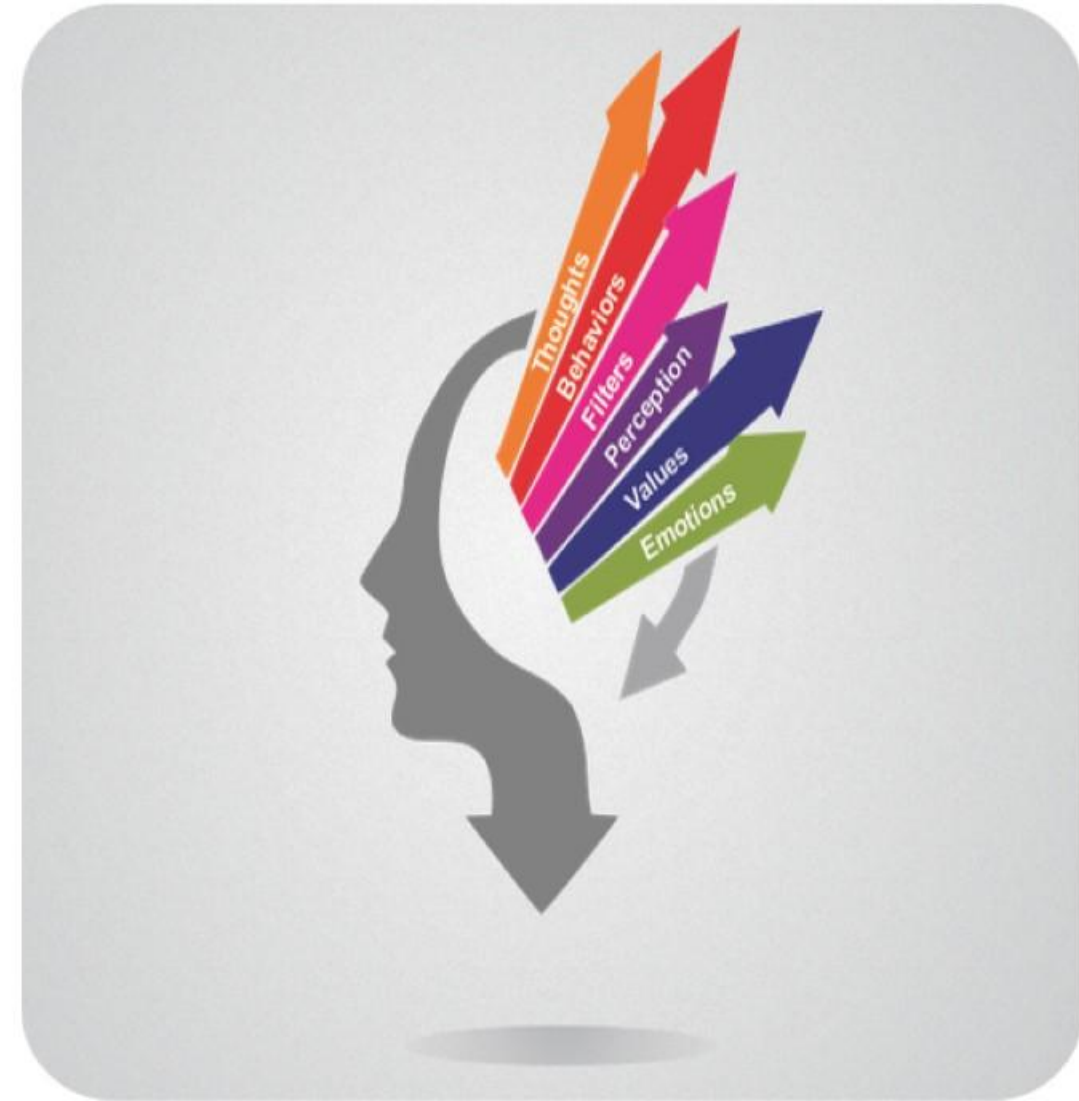


NLP
Foundation

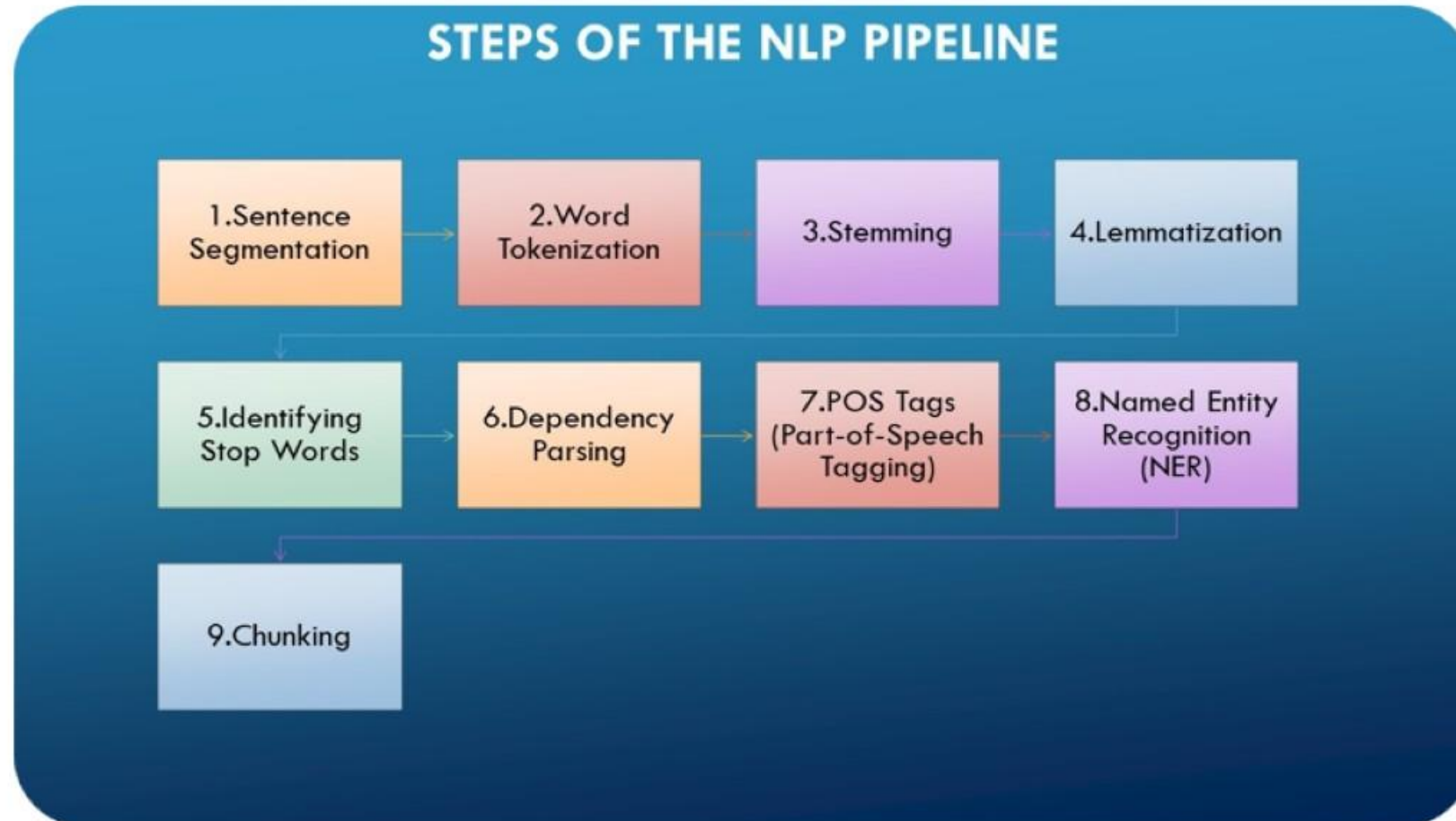
SESSION 2 – NLP FOUNDATIONS

NLP in Deep Learning

- NLP uses Deep Learning differently:
 - Works on text
 - Converts words → numbers
 - Uses embeddings
 - Used for sentiment, spam, customer feedback



NLP in Deep Learning



NLP Pipeline

- Text cleaning
- Tokenization
- Stopword removal
- Stemming / Lemmatization
- Vectorization
 - Bag of Words
 - TF-IDF
 - Word2Vec
 - BERT (advanced)

NLP Pipeline Example

- **1. Sample Text**

- text = "The quick brown foxes were jumping over the lazy dogs! They were extremely happy."

- **2. Text Cleaning**

- **Objective:** Convert to lowercase, remove punctuation, remove numbers, normalize text.

- **Code:**

```
import re
```

```
cleaned = text.lower()
cleaned = re.sub(r'^a-z\s', "", cleaned)
print(cleaned)
```

- **Output:**

- the quick brown foxes were jumping over the lazy dogs they were extremely happy

- **3. Tokenization**

- **Objective:** Split text into individual words (tokens).

- **Code:**

```
tokens = cleaned.split()
print(tokens)
```

- **Output:**

- ['the', 'quick', 'brown', 'foxes', 'were', 'jumping', 'over', 'the', 'lazy', 'dogs', 'they', 'were', 'extremely', 'happy']

NLP Pipeline Example

- **4. Stopword Removal**

- **Objective:** Remove the most common words that do not add meaning.

- **Code:**

```
import nltk
nltk.download('stopwords')
from nltk.corpus import stopwords
```

```
stop_words =
set(stopwords.words('english'))
```

```
filtered_tokens = [w for w in tokens if
w not in stop_words]
print(filtered_tokens)
```

- **Output:**

- ['quick', 'brown', 'foxes', 'jumping',
'lazy', 'dogs', 'extremely', 'happy']

NLP Pipeline Example

- **5. Stemming**

- **Objective:** Reduce words to root form (not always a real word).

- **Code:**

```
from nltk.stem import  
PorterStemmer  
stemmer = PorterStemmer()  
  
stemmed = [stemmer.stem(w) for w  
in filtered_tokens]  
print(stemmed)
```

- **Output:**

- ['quick', 'brown', 'fox', 'jump', 'lazi',
'dog', 'extrem', 'happi']

NLP Pipeline Example

- **6. Lemmatization**

- **Objective:** Reduce words to meaningful dictionary form.

- **Code:**

```
nltk.download('wordnet')  
from nltk.stem import  
WordNetLemmatizer  
lemm = WordNetLemmatizer()
```

```
lemm_tokens = [lemm.lemmatize(w)  
for w in filtered_tokens]
```

```
print(lemm_tokens)
```

- **Output:**

- ['quick', 'brown', 'fox', 'jumping',
'lazy', 'dog', 'extremely', 'happy']

NLP Pipeline Example

- **7. Vectorization Techniques**

- **A. Bag of Words (BoW)**

- **Code:**

```
from sklearn.feature_extraction.text  
import CountVectorizer
```

```
corpus = [text]  
vectorizer = CountVectorizer()  
bow =  
vectorizer.fit_transform(corpus)
```

```
print(vectorizer.get_feature_names_o  
ut())  
print(bow.toarray())
```

- **Output:**

- Vocabulary:
- ['brown' 'dogs' 'extremely' 'foxes' 'happy' 'jumping' 'lazy' 'over' 'quick' 'the' 'they' 'were']
- Vector:
- [[1 1 1 1 1 1 1 1 1 3 1 2]]

NLP Pipeline Example

- **B. TF-IDF**

- Term Frequency–Inverse Document Frequency

- TF = word frequency
IDF = how rare the word is
TF-IDF = importance score

- **Example:**

“free” appears in spam emails
more → high weight in spam

model.

- **Code:**

```
from sklearn.feature_extraction.text  
import TfidfVectorizer
```

```
tfidf = TfidfVectorizer()  
vector = tfidf.fit_transform(corpus)  
print(tfidf.get_feature_names_out())  
print(vector.toarray())
```

NLP Pipeline Example

- **8. Word2Vec (Using Gensim)**

- **Code:**

```
from gensim.models import Word2Vec
```

```
sent_tokens = [tokens]
```

```
w2v_model = Word2Vec(sent_tokens,  
vector_size=50, min_count=1)
```

```
print(w2v_model.wv['foxes'])
```

- **Output:**

- A 50-dimensional dense vector (e.g., [0.13, -0.24, ...])

- **9. BERT Embeddings (Advanced)**

- **Code:**

```
from sentence_transformers import  
SentenceTransformer
```

```
model = SentenceTransformer('bert-  
base-nli-mean-tokens')
```

```
embedding = model.encode([text])  
print(embedding.shape)
```

- **Output:**

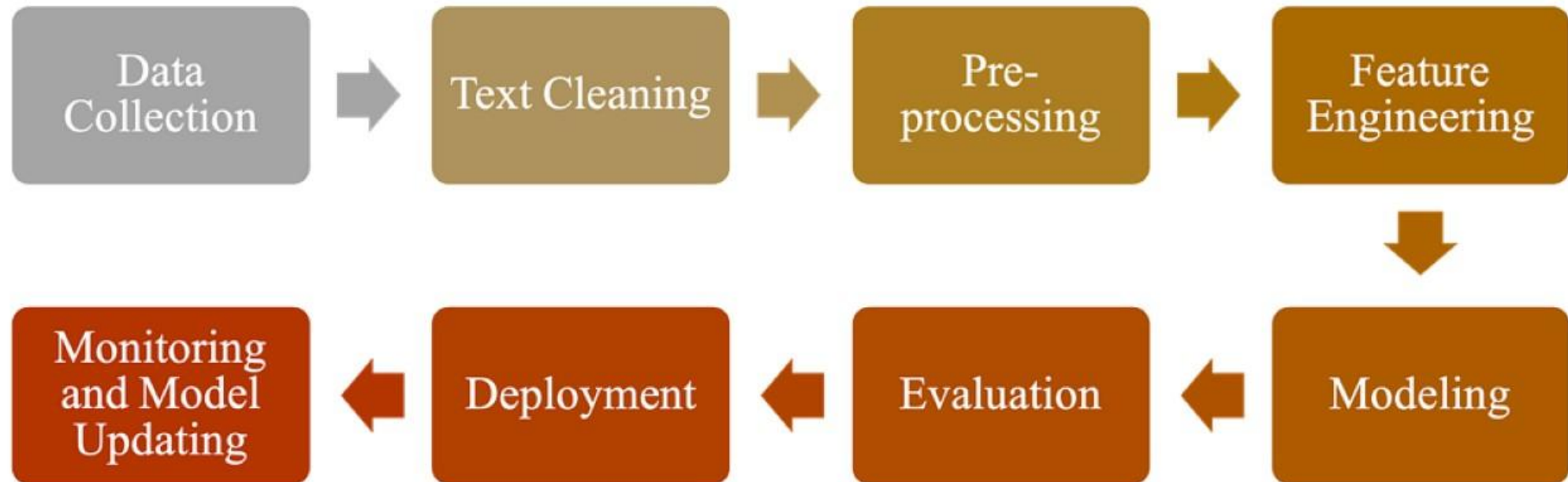
- (1, 768) # BERT produces a 768-dimensional embedding

NLP Pipeline Example - Summary

Technique	Purpose	Output Type
Cleaning	Remove noise	Clean text
Tokenization	Split words	List of tokens
Stopword Removal	Remove common words	Tokens
Stemming	Reduce to root form	Tokens
Lemmatization	Reduce to dictionary form	Tokens
BoW	Frequency-based representation	Sparse matrix
TF-IDF	Weighted text representation	Sparse matrix
Word2Vec	Semantic dense embeddings	Dense vectors
BERT	Contextual embeddings	768-dim vector

NLP in Deep Learning

NLP Pipeline



NLP Use Cases (Business)

Sentiment analysis

Complaints categorization

Spam detection

Chatbots

Ticket routing

Resume screening

Topic modeling

LAB 2A — Product Complaint Categorization (Text Classification)

- **Business Case**

- A consumer company receives thousands of product complaints.

- **Goal:** Automatically categorize into:

- Delivery Issue
- Product Quality
- Refund Request

- **Outcome**

- Automation of ticket routing in customer care.

- **Learning Outcomes**

- Build a complete NLP pipeline
- Text cleaning
- TF-IDF vectorization
- Train/test split
- Classification using Logistic Regression
- Predict category for new complaint
- Provide Westside-style policy suggestions automatically

 DAY 10 — AI Deployment,
Ethics & Capstone Prep

Day 9 - Recap

- Deep Learning Fundamentals
- Neural Network Architecture
- Activation Functions + Loss
- TensorFlow/Keras Essentials
- ANN for Classification & Regression
- NLP Foundations
- NLP Business Use Cases (Continued)

LAB 2A — Product Complaint Categorization (Text Classification)

- **Industry: Apparel Retail (Women, Men, Kids Wear)**
 - Use Case: Complaints received from customers on call centre / WhatsApp / email
- **Objective:**
 - Build an ML model that automatically classifies complaints into categories:
- **Main Complaint Buckets**
- **Delivery Issues**
 - Delay
 - Wrong item delivered
 - Missing items
 - Damaged during transit
- **Product Quality Issues**
 - Wrong size
 - Color fading
 - Color bleeding
 - Poor stitching
 - Material not as described
 - Fitting issue
 - Not matching the website image
- **Refund/Exchange Requests**
 - Refund request
 - Exchange request
 - Return pickup request
 - Credit Note asked (valid 6 months in premium stores like Lifestyle)

LAB 2B — WhatsApp Chat Spam Detection (Binary Classification)

- **(Spam Detection + Complaint Categorization + Sentiment Analysis)**
- **Business Case**
- A travel company (cab, bus, flight, hotel bookings) receives 1000s of WhatsApp messages daily:
- **Types of messages:**
 - Genuine customer queries
 - Complaints related to cab, bus, flight, hotel
 - Booking confirmation messages
 - Spam promotional messages
 - Angry customers asking for refunds
- **Build ONE NLP model that can classify:**
 - Spam vs Genuine Message
 - Travel Category: Cab / Bus / Flight / Hotel
 - Sentiment: Positive / Neutral / Negative
-  **Learning Outcomes**
- NLP preprocessing
- TF-IDF vectorization
- Multi-class classification
- Multi-output prediction
- Real-world text modelling

LAB 2B — WhatsApp Chat Spam Detection (Binary Classification)

-  **WHAT THIS LAB COVERS**
 - This lab is strong enough to represent actual **Travel Customer Support AI**.
- This is a **super-powerful** NLP lab covering:
 - ✓ Spam Detection (Binary Classification)
 - ✓ Travel Category Classification (Cab / Bus / Flight / Hotel)
 - ✓ Sentiment Analysis (Positive / Negative / Neutral / Spam)
 - ✓ Business Action Recommendation Engine
 - ✓ Real WhatsApp-type messages
 - ✓ TF-IDF + Logistic Regression models
 - ✓ Real-time model predictions

Why We Use **NLTK** in Natural Language Processing (NLP)

- **NLTK (Natural Language Toolkit)** is one of the foundational Python libraries for processing and analyzing human language data.
- We use NLTK because it provides **ready-made, industry-standard tools** for text preprocessing, linguistic analysis, and dataset preparation.
- In any ML/NLP pipeline, **cleaning and preparing text** is more important than the model itself. NLTK solves exactly that problem.

Why We Use **NLTK** in Natural Language Processing (NLP)

- Key Reasons Why NLTK Is Used

- **1. Text Cleaning & Normalization**

- NLP models work on structured, clean data. NLTK provides easy functions for:
 - Converting text to lowercase
 - Removing punctuation
 - Filtering unwanted characters
 - Tokenizing words
 - Without this, algorithms cannot interpret raw text properly.

-

- **2. Tokenization (Breaking Sentences Into Words)**

- Before any ML model can understand text, it needs to know:
 - What are the words?
 - Where do sentences start and end?
 - NLTK has highly optimized tokenizers such as:
 - word_tokenize
 - sent_tokenize
 - Regex tokenizers
 - This step is essential for all downstream NLP tasks.

Why We Use **NLTK** in Natural Language Processing (NLP)

• **3. Stopword Removal**

- Words such as "*the, is, am, are, of, and*" add no meaning for classification.
NLTK's stopwords list helps remove this noise easily.
- Example:
 - `from nltk.corpus import stopwords`

• **4. Stemming & Lemmatization**

- To reduce words to their root form:
 - **Stemming:** "working", "worked", "works" → "work"
 - **Lemmatization:** "better" → "good"
- NLTK provides:
 - PorterStemmer
 - WordNetLemmatizer
 - Models perform better when word variations are standardized.

Why We Use **NLTK** in Natural Language Processing (NLP)

- **5. Part-of-Speech Tagging (POS)**
 - NLTK can identify:
 - Nouns
 - Verbs
 - Adjectives
 - Adverbs
 - Useful for chatbots, sentiment analysis, and text generation.
 -
- **6. Named Entity Recognition (NER)**
 - NLTK can extract entities such as:
 - Names
 - Locations
 - Dates
 - Organizations
 - This is useful in travel chatbots, HR systems, ticketing systems, etc.

Why We Use **NLTK** in Natural Language Processing (NLP)

- **7. Corpora & Pretrained Datasets**

- NLTK includes many linguistic datasets such as:
 - WordNet
 - Movie reviews
 - Stopword lists
 - Tagged corpora
- These datasets help with both training and experimentation.
-

- **8. Teaching & Prototyping**

- For beginners and training programs, NLTK is ideal because:
 - It is easy to learn
 - Simple to demonstrate
 - Requires no heavy computation
 - Works well for small & medium datasets

NEXT NLP LAB — TEXT SUMMARIZATION + KEYWORD EXTRACTION (Advanced)

- This lab teaches:
 - Sentence tokenization
 - Keyword extraction
 - Frequency ranking
 - Automatic text summarization
 - Handling long customer messages
 - Creating executive summaries
- And it directly applies to:
 - Emails
 - Complaint tickets
 - Patient notes
 - Audit logs
 - Product reviews



LAB 2C — AUTOMATIC TEXT SUMMARIZATION & KEYWORD EXTRACTION

- **Business Case (Travel + Retail + Healthcare Ready)**
- Customer service teams receive long messages like:
 - Travel itinerary problems
 - Detailed complaints
 - Long email threads
 - Patient case notes
 - Flight delay grievances
- **EXTENSION** add:
 - Sentiment classification
 - Topic modeling
 - Auto policy recommendation
- Extract root cause (delay, rude driver, etc.)
- Managers cannot read everything.
 - We need an **AI summarizer** to extract:
 - ✓ Top keywords
 - ✓ Important sentences
 - ✓ A clean 3-line summary
- This gives a real “ChatGPT-like experience” using NLP fundamentals.

What we learned

- **✓ Keyword Extraction**
 - Shows the key problem signals.
- **✓ Automatic Summarization**
 - Gives 3-line actionable summary.
- **✓ Sentiment Classification**
 - Shows emotional intensity of the complaint.
- **✓ Topic Detection**
- Detects whether the issue is:
 - Cab Issue
 - Flight Issue
 - Payment / Refund
 - Service Quality
- Or combinations
- **✓ Auto Recommendation Engine**
 - Real business action plan:
 - Refund?
 - Driver behaviour action?
 - Voucher?
 - Rebooking help?
- **✓ User Input Mode**
 - Participants can test with live customer complaints.

 DAY 10 — AI Deployment,
Ethics & Capstone Prep

Agenda

1. Deployment Foundations

- What does “deploying a model” mean?
- Deployment models: Local, Cloud, API, Batch
- Key components needed to deploy

2. Model as a Service (MaaS)

- FastAPI / Flask architecture
- Real-time inference vs batch processing

3. Responsible AI & Ethics

- Algorithmic Bias
- Fairness
- Explainable AI (XAI)
- Legal & compliance aspects

4. Capstone Project Finalization

- Problem statement
- Dataset strategy
- Feature engineering
- Evaluation plan
- Deployment thinking

5. Week 2 Evaluation

What is AI Deployment?

- AI deployment means moving your trained ML model from a **development environment (Jupyter/VS Code)** into a **real operational environment** where real users, systems, or apps can use it.
- **Key Elements of Deployment**
 - Model
 - Code (API or Batch job)
 - Environment (local/server/cloud)
 - Input/Output validation
 - Monitoring & logging
 - Version control
 - Security

Deployment Options

- **1. Local Deployment**

- Model runs on a laptop/desktop
- Used for POCs, internal dashboards

- **2. Cloud Deployment**

- AWS, Azure, GCP
- Scalable but costly

- **3. API Deployment**

- Best for enterprise
- Used by apps, websites, CRM tools

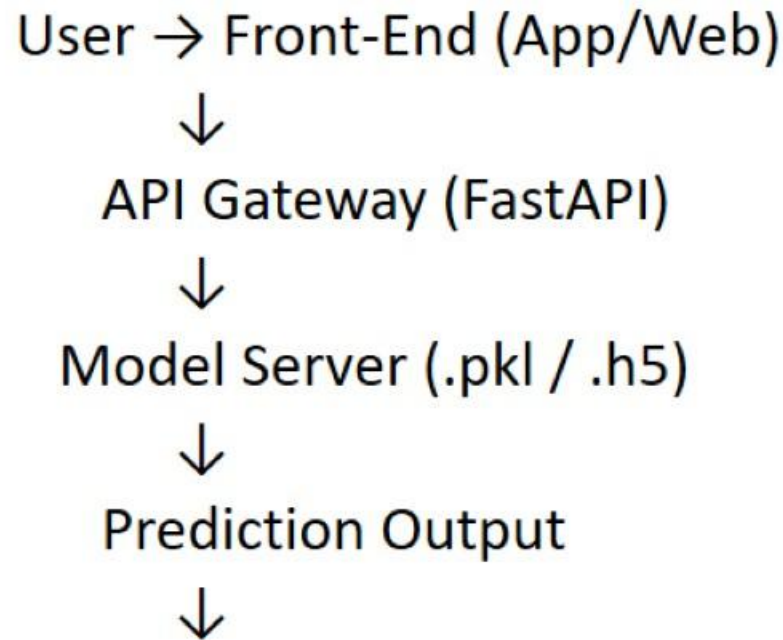
- **4. Batch Deployment**

- Large data processed at night
- Used in BFSI, Insurance, Retail MIS

API Deployment Architecture (FastAPI Example)

- Architecture Flow

Logging & Monitoring



Model Deployment Workflow

Train Model → Logistic/Tree/ANN/CNN

Save Model → pickle, joblib, .h5 (Keras)

Build API → FastAPI/Flask

Create predict() endpoint

Test using Postman / Curl

Deploy on server / cloud

Monitor predictions

Responsible AI

- Responsible AI ensures models are:

- Fair
- Transparent
- Safe
- Compliant with law
- Ethically aligned with business

- **Why Required?**

- Avoid harmful decisions
- Reduce bias
- Maintain trust
- Meet regulatory requirements (RBI, GDPR, DPDP Act India 2023)

Types of Bias

-  **Data Bias** – wrong dataset
-  **Sampling Bias** – skewed population
-  **Label Bias** – incorrect tagging
-  **Algorithmic Bias** – model disadvantages one group
-  **User Bias** – subjective human decisions

Fairness Metrics

- Demographic Parity
- Equal Opportunity
- Predictive Parity
- Treatment Equality

Simple example:

loan model rejecting more women with same credit score $\Rightarrow$ bias.

Explainable AI (XAI)

- **Why Explainability?**

- Business wants:
 - Why did model reject a loan?
 - Why did model mark fraud?
 - Why customer is predicted to churn?

- **XAI Tools**

- SHAP
- LIME
- Feature Importance (Tree/Forest)
- Partial Dependency Plot (PDP)

Legal & Ethical Risks

Data privacy (India DPDP Act 2023)

Lack of transparency

Unsafe predictions

Biased outcomes

Informed consent

Model misuse

Security breaches

Capstone Project Prep

Design an ML/AI solution covering:

- Business problem
- Dataset strategy
- ML model choice
- Training approach
- Evaluation
- Deployment approach

Capstone Problem Framework

- **1. Business Problem Definition**

- Industry
- Pain Point
- What needs to be predicted?

- **2. Dataset**

- Source
- Structure
- Key features

- **3. Feature Engineering**

- Cleaning
- Deriving new features

- **4. ML Model Selection**

- Logistic, Tree, ANN, KMeans, NLP etc.

- **5. Evaluation**

- Accuracy
- Precision-Recall
- F1 Score
- Confusion Matrix

- **6. Deployment Plan**

- API
- Cloud
- Dashboards

Capstone Examples to Choose

Insurance Claim Fraud Detection

Telecom Churn Prediction

Retail Demand Forecasting

HR Attrition Prediction

Credit Risk Modeling

Complaint Categorization (NLP)

Customer Segmentation (Clustering)

Week 2 Evaluation

MCQ + Business Case

Week 2 - Evaluation

1. Problem Statement

Hospitals face significant financial loss and operational pressure when patients return within **30 days** of discharge. Readmissions indicate issues in patient recovery, medication adherence, or missed follow-ups.

Your task is to **design, build, evaluate, and explain** an AI system that predicts:

- Whether a patient is at **risk of readmission**
- Why the model predicts the patient is high-risk
- What action the hospital should take

This assessment requires applying **all topics taught in Week 2**.

Thank You!